

Then the “movement” of an attribute from one block to a covering block is a matter of changing a few pointers. We leave the details to an exercise.

Exercises

1. Sketch a single-pass and a multiple-pass symbol table for the following multiple scope program. Show the table structure at the end of each block, just before the block exit occurs.

```
begin
  var ED, SAM;
  begin
    var MIKE,MARY,SAM;
  end;
  begin
    var ED,MIKE;
    begin
      var SAM,BILL;
    end;
    begin
      var AL,MARY;
    end;
  end;
  var SAM,BILL;
end;
```

2. In the multi-pass, multiple scope symbol table structure of figure 8.3, are reverse links between the attribute elements needed?
3. Consider Fortran data declarations. An identifier may appear in several different declarations in any order without error, as follows:
 - an EQUIVALENCE declaration
 - in a COMMON declaration
 - in a type declaration, e.g. INTEGER SAM,FRED(15)
 - in an ARRAY declaration, e.g. ARRAY MIKE(250)
 - in a DATA declaration

Furthermore, if an identifier is dimensioned in a type declaration, it should not appear at all in an ARRAY; if it appears in an ARRAY declaration and in a type declaration, it should be dimensioned in only

the ARRAY declaration. Give a set of attributes and rules for dealing with Fortran data identifiers. *Hint*—construct a finite state automaton that describes all legal declarations for an identifier.

4. Show informally that the symbol table structures illustrated in figures 8.2 and 8.3 guarantee that (1) an identifier declared in some block is available only within that block, and that (2) an identifier *I* declared in block *A* is unavailable in a nested block *B*, if *I* is also declared in *B*.
5. Suppose that a multiple pass compiler were to be implemented using the structure of figure 8.2. The multiple passes are only through the current scope. That is, as soon as a block end is reached, several more passes through the AST associated with that block are made to finish the compilation. Then the current block's symbol table can be deleted. Could such a system be made to work? Would there be any advantage, if so, over a single pass system?
6. In the single pass scheme of figure 8.2, the names are separated from their attributes in order to reduce the storage for the identifier strings. Suppose that name sharing is not attempted—a name could then be part of its attribute structure. The advantage of this is that a pointer from the attribute to the name is not needed; the disadvantage is that the same name may appear multiple times in the stack. Let an average name length be *N* bytes, and a pointer be 2 bytes. How much name sharing must occur in order for separated names to require less space than names associated with their attributes?

8.3. Data Objects and Their Static Representation

A *data object* is some entity associated with a name that comes into existence when the program that contains it is executed. It is carried by some memory field for the duration of its life, and it is operated upon by various program instructions. A given object may carry a variety of different values during execution and may reside in physically different sections of memory at different times. Its key property is continuity; its value is preserved between accesses and is only changed through some assignment statement in the program. Its life may or may not end when the program control passes out of its identifier's scope. Some data objects are created initially and survive for the life of the program, while others survive only while program control remains in the static scope of its identifier.

A data object has an external and an internal structure. Externally, its structure is defined by its language declaration—e.g. REAL or BYTE ARRAY. Internally, it is some field of binary information in memory,

possibly scattered about in the available memory space.

To a compiler, a data object is an entry in a symbol table. It possesses a set of attributes, and they may include a prescription of its location in memory during execution. The operations on a data object implied by the source language are partially determined by its attributes and result in a certain sequence of emitted code that, when executed, affect the memory cells assigned to the data object.

The treatment of the data object associated with an identifier depends on the attributes assigned to the object through its declaration. In most computer systems, the internal form of every data object is some sequence of binary digits which provides no clue as to its purpose. It is the task of the compiler to associate meaning with every data object and to guarantee that the specifications of the program with respect to the data object are faithfully reflected in the sequence of machine code being generated.

8.3.1. Primitive Objects

A *primitive data object* is an object that is normally treated as a unit by the operations of the language. It usually cannot be subdivided by any operation, and in many modern languages, the internal structure of the primitive objects is undefined.

Each data object is internally encoded in some manner that depends on the object machine system and the programming language system. For example, the 16-bit binary number

1110010100000010 or (octal) 162402

can be interpreted in any of the following ways on an HP3000 computer:

1. As the memory reference instruction STB Q+2, I (store a byte on the top of stack into a location marked by an indirect address located 2 words relative to the Q register).
2. As the "integer" - 6910 (decimal form).
3. As the "logical" 58626 (decimal form).
4. As an indirect address, pointing to a location 6910 words offset from the contents of the program register, "PB".
5. As the pair of bytes (8 bit data objects) 345 (octal) and 2. These bytes may be interpreted as the ASCII characters "c" and "STX", e.g., lower case letter "c" and the special character "STX".

One of these interpretations is selected during execution by an operation on the data. Thus the STB interpretation holds when that word is fetched into the instruction register and executed.

If the word is fetched and loaded in a data register, and then an "integer add" operation is executed, it is effectively interpreted as an "integer." If the

operation is a “logical multiply,” then it is interpreted as a “logical,” and so forth.

Every computer has a set of definitions of the primitive data objects upon which its instruction set is designed to operate. Matching the computer operations and data objects to those required by a source language is sometimes troublesome. For the sake of efficiency, as many of the machine operations as possible should be used for source language operations; if a difference in specification exists, then the language operation must be simulated by a procedure call or a sequence of machine operations. For most common languages, the common machine operations and language operations are arithmetic in character, making a match possible. However, different machines use different word lengths and representations, which affect the maximum range of numeric values. Some idea of the variety of number representation may be gained from the following examples.

1. On the IBM 1620, numbers are represented as a string of characters, representing decimal digits. Special marks are included for the representation of floating point numbers, signs, and the end of a number, since numbers may be of different lengths.
2. On the IBM 360, numbers may be represented in a variety of forms. There is a set of binary integer forms, 16 and 32 bits in length, short and long binary floating point numbers, and two decimal numbers (packed and unpacked). A small positive integer may be represented as a byte.
3. On the CDC 6000 series, only floating-point numbers 60 bits long are handled by the instruction set. The format is interesting, as there are representations for “infinity” and for “undefined.” Integer arithmetic is done with the floating-point hardware. The number format is 1’s complement, rather than the usual 2’s complement.

The representation of characters as an internal code is similarly nonstandard among the computer manufacturers. IBM has used EBCDIC, many smaller manufacturers use ASCII, and CDC uses its own kind of 6-bit character code, called *display code*, for character representation. Even within one manufacturer and one standard code, some variations are found, for example, ASCII has different versions to reflect differences in foreign languages.

Printers, terminals, and other character-oriented devices also exhibit coding variations, even within a single manufacturer’s product line.

Now consider the source language. What are its specifications with regard to its primitive data objects? Let us consider some example source specifications.

1. ANS FORTRAN (Campbell [1976]) says this of the “integer type” and “real type”:

“An integer datum is always an exact representation of an integer value. It may assume a positive, negative, or zero value. It may assume only an integral value. An integer datum has one noncharacter storage unit in a storage sequence. A real datum is a processor approximation to the value of a real number. It may assume a positive, negative, or zero value. A real datum has one noncharacter storage unit in a storage sequence.”

Of characters, the Fortran standard states “a character datum is a string of characters,” and that “a character datum has one character storage unit in a storage sequence for each character in the string.” The internal form of a character is left unspecified. The standard only addresses the issue of connectedness of a set of characters that constitutes a string.

2. In Algol 60 (Naur [1963]), three primitive objects (integer, real, Boolean) are provided. Very little is said about them except that integers and reals may be positive or negative numbers, and that Booleans may only assume the values TRUE and FALSE.

3. Pascal recognizes four primitive objects: integer, real, char, and Boolean. The report says of integer, for example, only that “it is an element of the implementation-defined subset of whole numbers.”

We see that the structure of the primitive data objects are not specified in these languages. Each of these languages is intended to be implemented on a variety of computers. The languages are also such that the internal details of primitive objects are not essential to an algorithm and should not be. If nothing in a program depends on the implementation details, then the program can very likely be transported to several quite different computer systems.

However, a compiler is implemented on some machine and has a particular object machine. It is usually desirable that the compiler, too, be as nearly machine-independent as possible, though it cannot entirely be so. The characteristics of primitive data objects enter a compiler in two ways: (1) some choice of data type must be made to handle conversion of source language constants, and (2) some section of the compiler must deal with conversion (if any) from a compiler “internal” form of constant to the object machine data form. Both these tasks clearly should be relegated to a few low-level procedures that can be easily modified if necessary for the sake of portability. The bulk of the compiler algorithm should be indifferent to the internal representation details demanded by any one host machine.

The permissible ranges of primitive data objects are subject to several limits: (1) those imposed by the source language (usually there are none), (2) those imposed by the compiler implementation (there should be none), and (3) those imposed by the object machine. Of these limitations, the last is the most crucial for performance. As many operations as feasible should exploit the machine hardware provided for the purpose, and these operations may function slightly differently on different machines.

Some languages provide means of specifying minimum number ranges or number properties. In Algol 68, a program may be preceded by a *standard prefix* that contains a minimum set of specifications that must be met by the operators and data objects of the program. An Algol 68 compiler for some target machine is expected to check this specification set against the target machine and report any difficulty in meeting the specification.

In Pascal, numeric data may be specified with a range, e.g.,

```
var day: 1..31; month: 1..12; year: 0..2000
```

The judicious use of a range, rather than just "integer," can overcome potential problems with numeric ranges and also make possible reductions in the size of the data structures.

In PL/I, the user may specify a base (decimal or binary), a scale (number of places by which a decimal point is assumed shifted), a precision (number of significant digits), and a mode (type). All of these ranges may be specified in a declaration, and the compiler is expected to select a target machine data object that meets or exceeds the source specification, and to perform whatever special operations that are necessary to make the machine data object operations conform to the source specifications.

Fortran provides no number range specification. Worse, standard Fortran permits quite general equivalences of one data object to another, e.g.,

```
DOUBLE D;  
REAL R;  
EQUIVALENCE (D, R);
```

With these, the data space associated with the double D and the real R are shared. Clearly, a determined (or naive) programmer can access the internal structure of a REAL (very machine dependent) in an otherwise high-level program.

8.3.2. Types

A *type* is a means within the source language of assigning a set of attributes to a data object. Most languages provide a set of standard primitive types, and some permit the user to organize a set of types and then to assign a type name to that set.

The assignment of a type to a data object is made through its declaration. Typing provides several benefits of considerable value:

1. The user often has a choice of more than one applicable type. He may then choose one that represents the best compromise between a needed range and memory economy.

2. A few operator symbols can represent a large number of operations. For example, the symbol “+” can represent the addition of a large variety of numeric objects, including pairs of differently typed objects. The specific operation implied by a “+” then depends on the types of its operands.
3. The compiler can provide a strong guarantee of operation integrity through the use of internal data conversions, warning messages, or error messages.

A *strongly-typed* language is one in which a typing of every object is required and is rigorously enforced. A compiler for such a language will never permit an object of some type to be operated upon as though it had another type, without an explicit command in the source language, or without a conversion.

The controls provided through data object typing are defeated if any of the following language features exist and are used:

1. Equivalences. An equivalence permits some target machine data object field to be shared among two or more objects, with possibly different types.

In Fortran, an equivalence can be created through the EQUIVALENCE statement or through the COMMON statement. Two different data objects thereby share a common data space. An equivalence of a Fortran REAL and an INTEGER will clearly permit some kind of access of the bits in the REAL as though they represented an INTEGER instead, without numeric conversion. Any program that exploits such a capability must have been written with a particular implementation in mind and is not transportable.

2. Binary operations on nonbinary types. For example, if the language permits a shift or a bit-by-bit logical operation on a real number, the real can be altered or accessed in ways that depend on its internal representation.

3. The use of characters or other nonarithmetic objects in arithmetic contexts. For example, the conversion of a string of digit characters to its equivalent numeric value requires a translation of each digit character to its equivalent value. It is best that this be done through a collation function, e.g., the Pascal function *ord(c)*. The function *ord(c)* returns an ordinal collation index for a character *c*, which is not necessarily the internal code for the character but which represents its alphabetic or numeric ordering.

4. Passing a data object by reference to a procedure call, when its type and the type of the corresponding formal object differ, without also passing type conversion information. Without conversion information, such a parameter pass can only be treated as an equivalence by the compiler.

5. Use of assembly language intermixed with the high level language.

Assembly language enforces no typing rules, and can be used to defeat any compiler system, but at the cost of total nonportability.

6. Writing a file with one format specification and reading it with another. The only protection against this possibility lies in a file label that carries type specifications for the file and a source language that permits only labeled files to be written or read.

7. Use of illegally large or small array indices to access some memory area not assigned to the array by its declaration. This very bad practice is in fact possible with most compilers, because array bounds checking can only be done with additional code, degrading performance. The deliberate use of an illegal array index must be condemned for any but the most primitive, machine-level programs. It presupposes that the adjacently accessed data areas will remain in place in the source, that anyone modifying the source code will understand and respect the practice, and finally that all future versions of the compiler will continue to allocate data object space in exactly the way the programmer expected. These suppositions are rarely true.

8. Uncontrolled use of pointers. A pointer is effectively a memory address. Its address value may be altered during execution and then used to access the information it points to. Pointers are valuable in assembly language, but have not found their way into many high-level languages, because of the difficulty of controlling their use.

Pascal is a notable exception since a pointer is always typed. It can only be declared in association with some type. When it is set, it may only be caused to point to a data object of its own type. Then when the pointer is used indirectly (i.e., to access the data object to which it points), the compiler and the programmer have confidence that the typing rules are not violated.

Type Conversion

A typical problem that arises in typed languages is mixed-mode type conversion, illustrated by the following Fortran example:

$$C = I + J * (R/D)$$

where I and J are type INTEGER, C is type COMPLEX, R is type REAL, and D is type DOUBLE PRECISION (floating point). The Fortran conversion rules are essentially expressed by a type hierarchy as follows:

COMPLEX > DOUBLE > REAL > INTEGER

This means that, given an operation on data objects of two types (t_1, t_2), that type t with the higher hierarchy is chosen as the type of the operation. The data object with the lower type is first converted to type t , the operation is performed in type t , and the resulting expression is considered to be of type t .

The type hierarchy in Fortran reflects an ordering imposed by the domains of the various types. Although the Fortran specification says only that an INTEGER or REAL datum has “one noncharacter storage unit,” and that a DOUBLE PRECISION or COMPLEX have “two noncharacter storage units” in consecutive order, it is reasonable to suppose that on most implementations,

- Every INTEGER value can be represented by some REAL, perhaps with some loss of precision for large integers.
- Every REAL value can be accurately represented by some DOUBLE.
- Every DOUBLE can be represented by some COMPLEX, perhaps with some loss of precision.

The hierarchy therefore provides a reasonable assurance that any binary operation allowed by the Fortran language will yield a valid result, although with some loss of precision in certain cases.

One binary operation is prohibited—between a DOUBLE-type and a COMPLEX-type data object.

The hierarchy is applied to each binary operation as it appears in an abstract-syntax tree as follows:

Let $h(t)$ be the hierarchy index of a type t , as expressed by the table:

type t	$h(t)$
INTEGER	1
REAL	2
DOUBLE PRECISION	3
COMPLEX	4

When the AST is constructed for some expression, each leaf is tagged with $h(t)$, where t is the type associated with the leaf. Then each operator node N is tagged with index $h' = \text{MAX}(h_1, h_2)$, where h_1 and h_2 are the hierarchy indices of N 's children.

For example, a node with a DOUBLE child and an INTEGER child would be marked DOUBLE.

The tree operations implied by these typing rules are of a bottom-up character; we say that the attributes for any node are *synthesized* from its descendant attributes. Clearly, the necessary node types can be constructed while the AST is being built by a bottom-up parser, assuming, of course, that all the leaf attributes are known during the expression parse (they are in most languages.)

An assignment statement in Fortran has somewhat different typing rules. Given the statement

$$v = e$$

a conversion from the “e” type to the “v” type is to be automatically generated by the compiler. A conversion is specified for every pair of types. The conversion from a smaller to a larger hierarchy (e.g., INTEGER to DOUBLE) preserves arithmetic value, though with some loss of precision in certain cases. However, those conversions from a larger to a smaller hierarchy may result in overflow or other undesirable side-effects.

Conversion is easily added to the AST in preparation for code generation. We merely add a unary conversion node between every pair of linked nodes of different type. This is most conveniently done during the bottom-up type decoration of the tree nodes. The assignment node is given the type of its left member, so that a conversion node (if required) will appear between the assignment node and its right member.

The assignment statement given above therefore yields the AST shown in figure 8.4, which carries four conversions. All the binary operations in the right-hand expression are performed in double precision, because of the structure of the expression. If the “*” were to be replaced by a “-”, then the $I + J$ would be performed in integer mode and converted to double precision for the subtraction.

The Fortran standard rules illustrate a typing system used in several languages. Other typing systems have been defined for special languages. For example, a Fortran implementation on a CDC 6000 series system has the following typing rule for expressions:

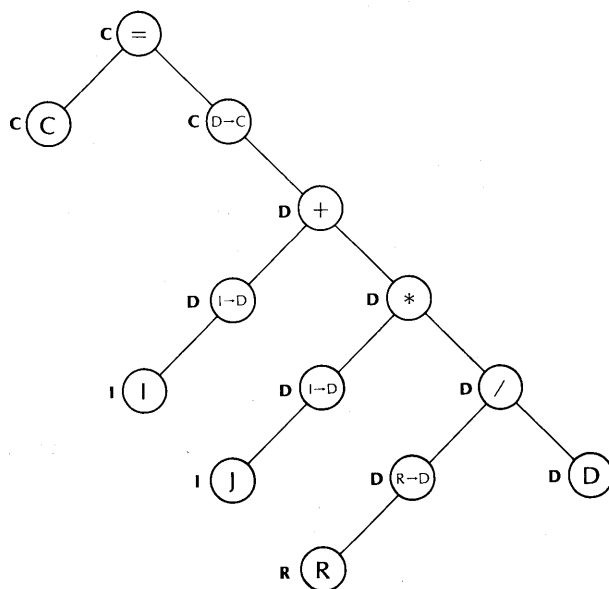


Figure 8.4. A typed AST, with conversions added, for the Fortran assignment statement $C = I + J * (R / D)$. The notation $R \rightarrow D$ means “real-to-double conversion.”

Every operation covered by a pair of parentheses, or every operation of an open expression, is performed in the same type. That type is hierarchically the largest of the types in the set of operands associated with the operators.

A pair of operations is said to be *covered* by a parenthesis pair if the parentheses are matched, and they are either not separated by any parentheses or are separated by a matching pair of parentheses. An *open* expression is one not enclosed by any parentheses.

Thus in the following expression all the $+$ operators are executed in type REAL:

$$(I + I - (R * I) + I)$$

The REAL expression in the inner parentheses causes all the I operations to be performed in type REAL.

We leave as an exercise the construction of a type-decorated AST with inserted conversions for these typing rules.

Exercises

1. Construct a typed AST for each of the following Fortran assignment statements. I , R , D , and C refer to data objects of the four Fortran types. Which of them contains a typing error?

$$R = I * C - \text{COMPLEX}(D/I)$$

$$C = I * C + I/R * (D + R)$$

$$D = R + (I * (I - I))$$

$$I = C + C - I * R / I + R$$

2. Give an algorithm for the construction of a type-decorated AST for the CDC 6000 Fortran rules. Notice that parentheses disappear in the AST—what can be done to retain their influence on the node typing?
3. Give an example of an expression that will evaluate differently under ANS Fortran and CDC Fortran typing rules.

8.3.3. Structures

User-defined structures provide powerful means of organizing algorithms. The design and expression of a suitable data structure is as important as the design and expression of the program statements.

A *structure* is any assemblage of primitive data objects organized in such a manner as to facilitate the coding of some algorithm. As a program might be structured through the use of if-then-else or while-do statements and

begin-end blocks, so might data be similarly organized. An optimally clear and concise program results from the balanced use of control and data structures.

The most primitive assemblage of data is probably the one-dimensional *array*, or *vector*. An array permits the selection of any one object of a set by means of an integer index. Each of the array elements have the same type and are normally arranged in contiguous memory locations. Whether an array must be arranged contiguously depends on the language. Fortran permits rather general equivalences, hence the arrangement of array elements in memory is an important language specification. PL/I guarantees contiguity only if the array is part of a "structure" and not otherwise.

An obvious extension of an array is a *multidimensioned array*, or *matrix*. At one time, Fortran permitted matrices with no more than three dimensions. In Fortran IV, an array may carry any number of dimensions. Algol 60 permits arrays of any number of dimensions, as does Pascal and PL/I.

Arrays may be statically or dynamically dimensioned. A statically dimensioned array carries bounds fixed at compile time. A dynamically dimensioned array carries bounds that may be dynamically adjusted when control passes through its declaration, or when a special allocate statement is executed. Dynamic arrays are usually more costly to implement than static arrays, but permit greater control over the use of memory resources and greater flexibility in programming.

A more general data structure is the PL/I *structure*, which is a tree of data objects. Each of the objects is a tree leaf and may be accessed through a special name convention from the root down. The nodes may be arrays. The purpose of a PL/I structure, beyond documentation (the separate pieces could just as well be declared separately), is to collect disparate data objects together in a contiguous section of memory and to permit all or some of the data objects in a PL/I structure to be accessed as a unit through one name reference.

Another form of data structure is the Pascal *user-defined type*. Here, a data structure may be given a name without an accompanying allocation of data objects. That name may then be used in its own structure, or in other structures, and will ultimately represent some set of data objects. For example, a type that contains its own name can express a quite general directed graph in a compact form.

The issues of data structuring as they apply to compiler design are these:

1. How might a data structure's static properties be represented internally as a symbol table structure?
2. Given a reference to some data object, perhaps a member of some structure, how can its symbol table entry be located? As we shall see, this is not a trivial problem for PL/I and Cobol.
3. How might a data structure be represented at run-time?

4. Given a data structure's representation, how is a data object best accessed?
5. If the data structure is dynamic, what run-time representations and access mechanisms are required? Several different kinds of dynamic structures have been proposed in different languages, and they require rather different run-time representations.

We shall deal with the static issues (1,2) in this section, and deal with some of the dynamic issues (3,4,5) in the next chapter.

8.3.3.1. Array Objects

The most common aggregate of primitive data objects is the array. Arrays are accessed through some base address and one or more integer indices. Whether an array is physically in contiguous memory is important in Fortran, but not in Algol, Pascal, or PL/I. It is usually only necessary that a mapping from a set of indices to a unique element exists. However, we shall develop two access methods; the first requires contiguity, and the second requires contiguity only of each slice of a multidimensional array.

An array may be static or dynamic. A *static* array has a set of bounds fixed at compile time. These bounds are carried in the symbol table as part of the array's attributes, and used to generate code to compute an effective index for every reference. We shall deal only with static arrays here. Dynamic arrays are treated in chapter 9.

Suppose an array is dimensioned

```
var A: array[b1..u1, b2..u2, b3..u3] of integer;
```

and we have a reference

$$A[e_1, e_2, e_3]$$

The b_i and u_i are compiler constants. The e_i are expressions to be computed at run-time in general. Now the integer values of e_1 , e_2 , and e_3 must be within the dimensions, i.e.,

$$b_1 \leq e_1 \leq u_1$$

$$b_2 \leq e_2 \leq u_2$$

$$b_3 \leq e_3 \leq u_3$$

A reasonable compiler assumption is that these constraints are always met. If desired, the compiler may emit additional run-time code to check each of the array reference indices e_1 , e_2 , e_3 against the required limits.

The array A may be mapped into linear memory in several different ways. We present two here, suitable for static arrays, and a third, for a dynamic array, in the next chapter.

Linear Mapping. The array A may be mapped onto a contiguous linear memory space in increasing order as follows:

$$\begin{array}{l}
 A[b_1, b_2, b_3] \\
 A[b_1, b_2, b_3 + 1] \\
 \vdots \\
 A[b_1, b_2, u_3] \\
 A[b_1, b_2 + 1, b_3] \\
 A[b_1, b_2 + 1, b_3 + 1] \\
 \vdots \\
 A[b_1, u_2, u_3] \\
 A[b_1 + 1, b_2, b_3] \\
 \vdots \\
 A[u_1, u_2, u_3]
 \end{array}$$

Let the address of the first of these be denoted B. Then the offset X of any other element, with indices e_1, e_2, e_3 , is clearly

$$X = n_3 * n_2 * (e_1 - b_1) + n_3 * (e_2 - b_2) + e_3 - b_3$$

where

$$\begin{array}{l}
 n_2 = u_2 - b_2 + 1, \text{ and} \\
 n_3 = u_3 - b_3 + 1.
 \end{array}$$

Note that $n_1 = u_1 - b_1 + 1$ is unnecessary and that X may be rewritten as

$$X = n_3 * (n_2 * e_1 + e_2) + e_3 - (n_3 * (n_2 * b_1 + b_2) + b_3)$$

The last term is a compile-time constant, while the first two must be evaluated at run-time. This expression for X suggests that we should

1. Create a fictitious "base" address

$$B' = B - (n_3 * (n_2 * b_1 + b_2) + b_3)$$

2. On a compiler reference, compute an index

$$X' = n_3 * (n_2 * e_1 + e_2) + e_3$$

(two additions and two multiplications).

3. Access the element as

$$\langle \text{operation} \rangle B', X'$$

i.e., an indexed reference through a base address B' .

The generalization of this scheme to any number of dimensions should be clear.

The symbol table attributes associated with A are also clear—we need only the fictitious base address B' and the parameters n_2, n_3 .

Exercises

1. Generalize the linear access scheme to n dimensions, where $n \geq 1$.
2. Matrix elements are usually accessed in a loop, with index values incremented in unit steps on each pass through the loop. Discuss informally some optimizations that could profitably be applied to such loops. For a case study, consider Warshall's algorithm given in chapter 5, or matrix multiplication.
3. Suppose the target machine is a stack machine. Design instructions that carry out as much of the array access as possible, leaving a minimum to be executed by compiler-generated instructions. Are fewer instructions required?
4. Discuss array bounds checking at two levels—(a) each index must be in range of its own dimensions, and (b) the final referenced datum must be in range of the A matrix values. Are there sound reasons for preferring the first despite its obviously higher cost?
5. Suppose that indexing arithmetic is limited to a finite word size, e.g. 12 bits. Discuss the problem of trapping all illegal indexing operations at compile time, given various kinds of lower and upper dimension limits. Is it possible that every dimension and reference is within range of the number size, but that some intermediate result is out of range?
6. Suppose one or more of the index expressions e_i are compiler constants. How might the number of operations required to produce an indexed reference be reduced?

Matrix Pointers. Another way of accessing a multidimensioned array is through a system of matrix pointers. For example, consider a two-dimensional array

```
var X: array [0..m, 0..n] of integer;
```

We construct a vector V of size $m + 1$, containing pointers. Each pointer is directed to a different row (second dimension) of the two-dimensional array. Then the access of an element $Z[i, j]$ is a matter of fetching $V(i)$, which points to an array slice, then indexing into this slice through index j . On certain machines, this combined operation can be very fast—we only need a set of index registers and a special operation that steps the machine through the pointer arrays to the final data array element.

The fetching of $Z[i, j]$ might appear as follows in stack machine code:

```

LOAD i;    {evaluate first index}
STAX;      {move value to index register; TOS is deleted}
LOAD V,X;  {fetch pointer to the i'th slice}
LOAD j;    {evaluate second index}
STAX;      {move to the index register}
LOAD TOS,I,X; {fetch the value indirectly through
               pointer on stack and index j}

```

Exercises

1. Develop a formula for the memory space required for an array of n dimensions $d_1, d_2, \dots, d_n$. Assume that each pointer requires one unit of storage and that each data element requires b units of storage. Then express the percentage overhead required for the pointers and show that this relative overhead decreases to zero as the dimensions increase. (The parameter b is usually 1 or 2.)
2. Discuss the relative efficiency of the matrix pointer and linear access methods, with and without optimizations. Examine a specific algorithm, e.g., Warshall's algorithm or matrix multiplication.
3. Assume the target machine is a stack machine. Design high-level instructions to retrieve and store a data object organized by matrix pointers. Should a pointer have any special flags, e.g., indirection?

8.3.3.2. PL/I Structures

Several modern languages provide means of declaring collections of data organized as lists and/or trees. One of the earlier aggregate declaration forms is the PL/I structure, illustrated next. (Cobol has a similar structure definition.)

```

DECL 1 PAYROLL(75),
      2 NAME,
      3 LAST CHARACTER(12),

```

```

53 FIRST CHARACTER(8),
  3 MIDDLE CHARACTER(1),
2 PAY_NO CHARACTER(5),
2 HRS,
  3 REGULAR FIXED DECIMAL(2),
  3 OVTIM FIXED DECIMAL(2),
2 RATE,
  3 STRATE FIXED DECIMAL(3,2),
  3 OVRTIM FIXED DECIMAL(3,2);

```

This structure defines a payroll file for a firm with 75 employees. We have a static tree structure with the leaves: LAST, FIRST, MIDDLE, PAY_NO, REGULAR, OVTIM, STRATE, and OVRTIM. The remaining names are internal nodes. Each of the leaves is an elementary data object or an array. Each of the interior nodes can be an array, and if so, means that its subtree is replicated in memory by as many elements as specified in the array dimension.

The integer prefixes define the *level number*; the syntax does not otherwise fix the tree level numbers.

A data element is accessed through a composite name, e.g.,

PAYROLL(15).HRS.OVTIM

or

PAYROLL(P).NAME.LAST(X)

The latter name designates character X of the last name of employee P.

A set of data objects may also be accessed through the name of some internal node in the structure. For example, PAYROLL(2).NAME stands for the set of three data objects LAST, FIRST, and MIDDLE of the second employee.

The name

NAME.HRS

is illegal; the two parts NAME and HRS are drawn from the same level.

PL/I permits abbreviating a structure element name, by omitting any of its components, provided that the specified data object is unambiguously designated. Abbreviation poses an interesting and nontrivial problem of symbol table access, and poses a need of detecting ambiguous name references, which we shall deal with shortly. Array indices may also be moved to the right, but may not be reordered or omitted. Because of index movement, every array reference must carry an index. For example,

PAYROLL.NAME.LAST(P)(X)

and

LAST(P)(X)

are also legal representations for the name PAYROLL(P).NAME.LAST(X)

The remaining features in the PL/I PAYROLL structure given above are attributes of the leaf node data object:

- CHARACTER(12) is an array of 12 characters.
- DECIMAL(2) is a decimal number with 2 significant places, e.g., any number in the range 00-99.
- DECIMAL(3, 2) is a decimal number with a decimal fraction, 3 digits ahead of and 2 behind the decimal point, e.g., 345.67.

A picture of part of this structure as it might appear in memory at runtime is given in figure 8.5. Only one of the 75 PAYROLL subtrees (the one for index 5) is broken out and completely displayed. For example,

PAYROLL(5).HRS.REGULAR

has the value 40 according to this structure.

PL/I Name Scanner. A PL/I structure name may be completely or partially specified. Any partial specification (consisting of only some of the names along a path from a root to the data object leaf) is legal if a unique data object is thereby specified.

A name in PL/I, whether partial or full, is considered ambiguous only if, as

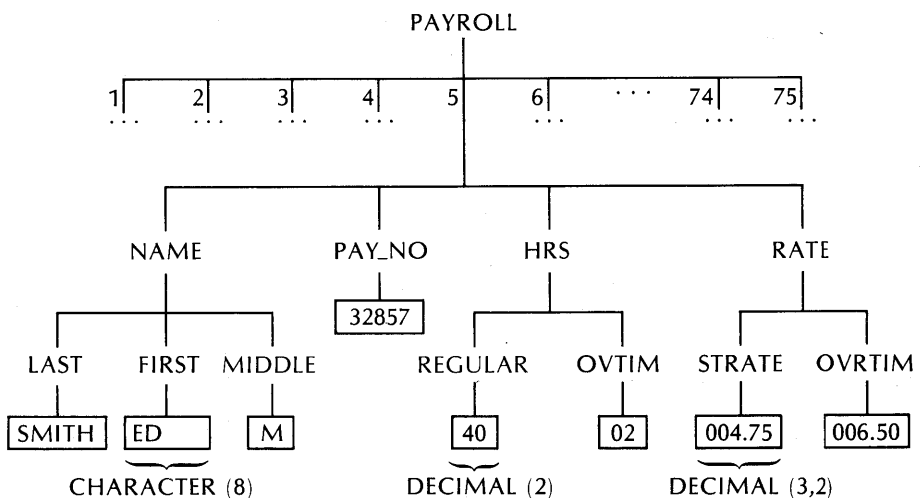


Figure 8.5. A PL/I structure displayed as it is organized at run-time.

a full name, it refers to two or more data objects. (Note that a “data object” may be a set of primitive objects in a structure.) If some name *N* can refer to two different objects, as a full name for one object *A* and as a partial name for another object *B*, then it is assumed to refer to object *A*, that is, a full name overrides any possible partial names.

It is also possible that the structures of a PL/I program are such that some data objects cannot be unambiguously accessed. If no reference to one of these data objects exists in the program, then the structures are acceptable, despite their inherent ambiguity. We see that any structure is permissible *per se*; however, each of the name references must be examined for possible ambiguities.

A clever solution to this name abbreviation problem was proposed by Gates and Poplawski (Gates [1973]). It is an interesting application of finite-state automaton methods.

A FSA is constructed with these properties:

1. Each transition is on a primitive name that appears somewhere in a PL/I declaration. For example, *NAME* is a primitive name in the structure example given above.
2. The halt states indicate either legal or illegal composite symbols. An illegal composite name is ambiguous—it could refer to two or more different data objects, possibly in different structures.

The construction of the FSA is along the following lines. Details and a worked-out example may be found in Gates’ paper; also see Abrahams [1974] for additional remarks. Knuth [1968] also gives a Cobol name resolution algorithm.

1. Construct a list of partial names. Each of these is a composite name formed by starting at the root of some structure and following a path part way toward a leaf. Each name contains at least one primitive name. The full names are in this list.

2. Associate a state with each name in the list. Add a unique start state.

3. Each transition in the FSA is on a primitive name. Each sequence of transitions beginning with the start state and ending in a halt state must spell out a fully qualified name. For example, if a full name is *A.B.C*, then there will be a sequence of transitions:

from	on name	to
S	A	A
A	B	A.B
A.B	C	A.B.C

Thus recognition of the full name *A.B.C* ends in the state associated with that name.

Clearly, if no ambiguities exist and all names are fully specified, this machine will accept the structure names of the language. However, these conditions are not always met.

4. To deal with partial names, we consider one path P in the machine from the start state S to a halt state. Add all the possible transitions permitted by the following rules:

- (a) A transition from state Q to R will be added if a directed path P' from Q to R already exists, and if P' is in P.
- (b) A transition to a state with nameX will carry the primitive name X.

Also make every state other than S a "new" halt state.

5. We have constructed a machine that is possibly nondeterministic. Resolve its nondeterminism by introducing composite states. Assign new numbers to the composite states. These states are to be distinguished from the existing states.

6. Repeat steps 4 and 5 for all the machine paths to yield a new deterministic FSA. It will have "old" halt states and "new" halt states. The "new" halt states represent sets of old and/or new states. This machine will accept any abbreviated composite name compatible with the declared structures. If it ends in a "new" halt state, the name may be ambiguous; there could be two or more distinct data that could be so designated (see rule 7). If it ends in an "old" halt state, the name is unambiguous; furthermore, the state is associated with the complete data object name.

7. A "new" halt state represents an ambiguous name if it contains two or more legal full names.

Of course, if no transition on some primitive name exists in the machine, then the composite name cannot be a part of any data structure and is thereby illegal.

Exercises

1. Construct an FSA for the following PL/I structure. Which partial name references are ambiguous? Which are illegal?

```
DECL 1  A,
      2  B,
      3  C BINARY,
      3  D  BINARY,
      2  C,
      3  D,
      4  E  BINARY;
```

2. Write a grammar for PL/I structures, at least the level and name parts. The primitive type specifications are complicated and may be omitted or generalized. Then assume that space for the structure is allocated in linear, contiguous memory. Develop an algorithm for computing the offset of any data object, and finally an algorithm for accessing any data object, given the necessary indices.

8.3.3.3. Pascal Structures

In Pascal, structures are created through the use of statements called *type* declarations. A type declaration specifies a data structure, but does not allocate memory space for it. There are several forms, but the one we are interested in is

```

type T = record S1 : T1;
                S2 : T2;
                S3 : T3;
                .
                .
                .
                Sn : Tn
end

```

The S_i are called *selectors*. The T_i are either other type identifiers or data object declarations, e.g., a simple object or an array. An array of elements of type Z is written thus:

array [3..9] of Z

The data structure represented by the type T is shown in figure 8.6.

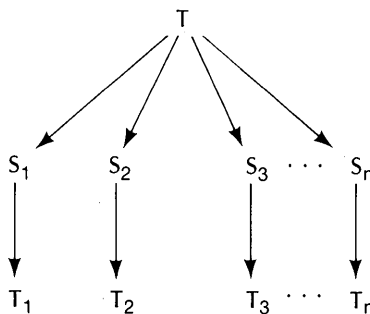


Figure 8.6. Compile-time data structure of a general Pascal RECORD declaration. T and T_i stand for type names, and the S_i are selector names.

Let us illustrate the Pascal convention with an example, and then we shall describe the kind of compile-time structures needed. This example defines a personnel data structure for Consolidated Widgets, Inc., or “CONSWIDGET” for short. The officers of the company enjoy their own section of the data structure, but share their VITA with all the other personnel. Note that it is necessary to define a given kind of data structure only once; it may be shared by any other structure as needed.

```

type NAMEPART = array[1..20] of char;

type NAME = array [1..3] of NAMEPART;

type STREETADDRESS = record STREETNUM : integer;
                           STREETNAME : NAME
                        end;

type VITA = record NAME : NAME;
                  AGE : integer;
                  SS : array[1..3] of integer;
                  SALARY : integer;
                end;

type OFFICER = record TITLE = (pres, vp, secy, treas,
                              boardmember, boardchairman);
                  VITAE : VITA
                end;

type ADDRESS = record STREET : STREETADDRESS;
                  POBOX : integer;
                  APT : integer;
                  CITY : NAME;
                  ZIP : integer;
                end;

type COMPANY =
  record NAME : NAME; {the two NAMEs mean
                      different things—this sort of
                      name conflict is legal}
        HDQTRS : ADDRESS;
        OFFICERS : array[1..N] of OFFICER;
        PERSONNEL : array[1..M] of VITA
    end;
```

So far, no data space will be allocated for any of this structure. The following Pascal declarations create data spaces corresponding to the designated type:

```

var PERSONNELFILE : PERSONNEL;
    {creates a data object of personnel vitae}

var COMPANY : CONSWIDGET;
    {creates a data object of an entire company,
     Consolidated Widgets, Inc.}

var LEONARD : VITA;
    {creates a data object with name, age, social
     security number, etc.}

```

An example of a LEONARD data object is shown in figure 8.7. It consists of three NAMEPARTs, each an array of 20 characters, an integer AGE, an array of three integers SS and an integer SALARY.

It is clear that the “COMPANY” type structure, when expanded, is quite large, yet its description and its manipulation (as we shall see) can be very concise.

A reference to a Pascal data object is through a composite name, similar to that used in PL/I. For example, LEONARD’s last name is accessed by

LEONARD.NAME[3][I]

where the I subscript indicates the specific character in the name. Pascal has

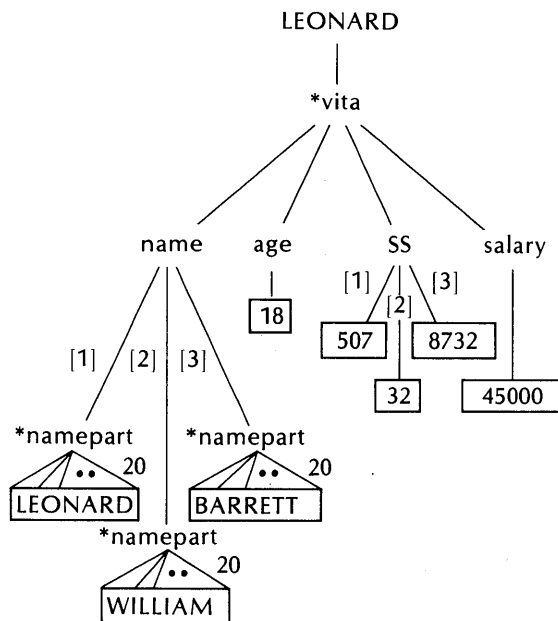


Figure 8.7. Example run-time data structure for a VITA structure.

no operations on a set of primitive data objects, as does PL/I and Cobol. Every name must refer to a primitive object.

The pair of subscripts [3][I] may also be written [3, I]. However, the subscripts may not be moved to the right; we shall see why not shortly.

The names may be abbreviated, but not in quite the same way that PL/I permits. Pascal provides a structured means of supplying partial names, the *with* statement. An example follows.

```

with LEONARD do
begin
  var I,J: integer;
  for I:=1 until 3 do
  begin
    J:=1;
    while NAME[I, J] <> ' ' do
    begin
      print(NAME[I, J]);
      J:=J+1
    end;
    print(' ');
  end
end

```

The two occurrences of NAME in this program are effectively preceded by “LEONARD.” We shall see that this is more than just a convenience for the programmer, it can also reduce code and execution time if several operations on the elementary data objects appear in the “with” block.

The general form of a “with” block is:

```

with <name-list> do
begin
  ...
end;

<name-list> ::= <name-list> , <composite-name>
              ::= <composite-name>

```

The composite names in the <name-list> may contain indexing or whatever, including indices computed during execution. Thus in the CONSWIDGET data structure a legal “with” block is

```

with CONSWIDGET.OFFICERS[X+3]. VITAE do
begin

```

```

NAME:= . . . ;
SS := . . . ;
.
.
.
end;

```

Pascal “with” blocks may also be nested, and an inner one may particularize a variable name heading started by an outer one, e.g.,

```

with CONSWIDGET do
begin
.
.
.
with OFFICERS[X+3] do
begin
.
.
.
with VITAE do
begin
.
.
.
end
end
end
end

```

Pascal Type Symbol Table Structure. We now discuss the data structures internal to a Pascal (or similar) compiler that will support the “type” and “with” structures.

A starting point is the so-called “type structure,” illustrated in figure 8.8, for the CONSWIDGET type given earlier. This is a directed acyclic graph (DAG) decorated with a set of attributes as shown. Each of the type names is marked with an asterisk (*); these names will not appear in a composite name. An elementary data object type is marked with a dagger (†). An array of objects or types is indicated by an “array” node, for example,

array [1..3]

The linkage in figure 8.8 clearly reflects this set of type declarations, and it can be constructed through productions and DAG-building semantics on the productions by methods discussed in chapter 7. We shall see that each of these links is necessary, including the link from the right-most child of a type node to its parent.

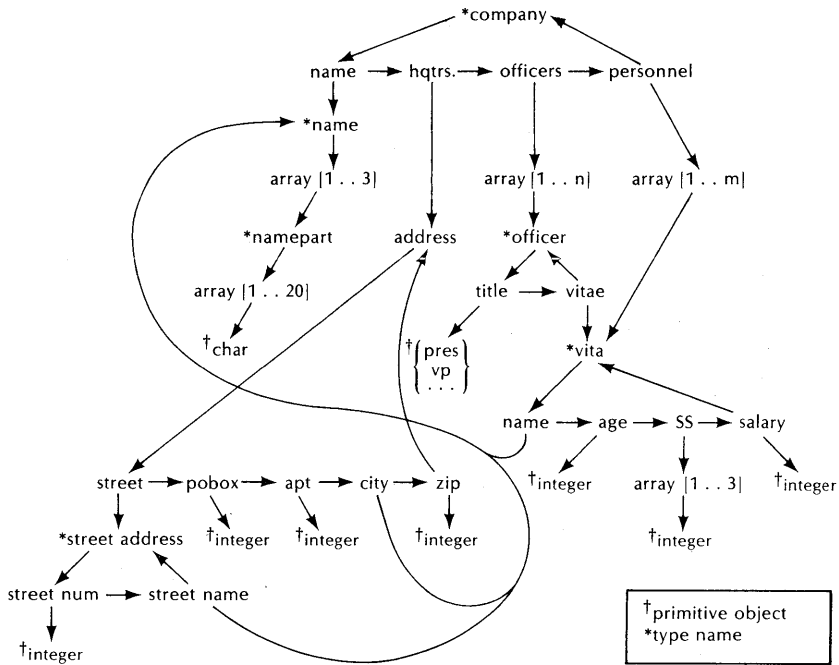


Figure 8.8. The compile-time structure for the COMPANY type structure.

The size of this structure is proportional to the number of statements in the type declarations; there is no expansion of data arrays, etc.

In addition to the links shown, there will also be a set of links from an access system to each of the simple names, to facilitate the rapid location of a given name.

We have several instances of identifiers with two or more uses, e.g., NAME. Pascal permits duplicate name use provided that no ambiguity exists. This feature causes some trouble in searching for names, especially in a “with” block, which we shall discuss later.

Inside a “with” block, a *with list* will be nonempty. Its form is shown in figure 8.9, simply a linear, linked list of nodes that indicate “type” nodes in the type structure.

The “with” list shown in figure 8.9 corresponds to the nested “with” blocks:

```
with CONSWIDGET do {creates the “*company” node}
begin
```

```

.
.
.
with HQTRS.STREET do {creates the
    "*streetaddress" node}
begin
    {with list has the form figure 8.9 here}
end;
    {drop the "*streetaddress" node here}
end
    {empty the with list here}

```

Note that the new "with" list nodes are added at the top level and are removed last-in, first-out.

Now consider the problem of locating a composite name in the symbol table structure. Suppose first that a fully qualified name is given, e.g.

CONSWIDGET.HQTRS.STREET.STREETNAME[2, 5]

We need locate only CONSWIDGET through the name access system, making sure that we locate a data name, not some other kind. That entry should point into the type structure, and we simply follow through the structure by comparing selector names only. In the process, we note array dimensions. Eventually, this process must lead to an elementary object.

Suppose instead that we are inside the inner "with" block given above, covering the head CONSWIDGET.HQTRS.STREET, and we are confronted with the variable name

STREETNAME[2, 5]

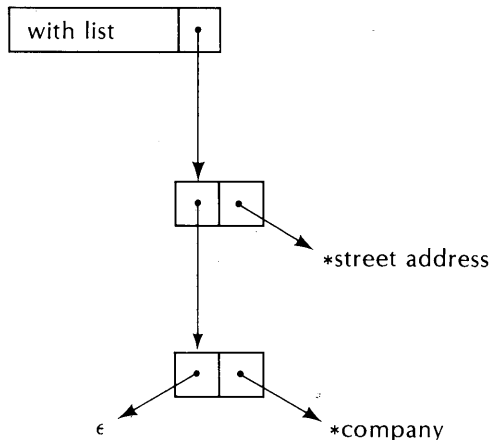


Figure 8.9. An example WITH list.

The name access system can locate a STREETNAME (maybe several, if we kept looking); we find that STREETNAME is not a data name, but a selector name. We know its location in some type structure, and as a selector, it must have a “type” parent. We locate the parent, using the reverse link from the right-most sibling to the parent, and the sibling links shown in figure 8.8. This process yields the type name “*STREETADDRESS”. We then search the “with” list for the pointer value of “*STREETADDRESS” and of course find it in the topmost cell (see figure 8.9).

If we had instead the partially qualified name

OFFICERS[5].TITLE

then the second node in the “with” list would match the type pointer “*COMPANY”.

Now Pascal permits name conflicts. We must therefore alter our strategy somewhat. If the above search of the “with” list fails to yield a match, then we may be looking at the wrong identifier in a name chain, i.e., we could be looking at a selector or type name that conflicts with some other valid name. We therefore must repeat the search process with the next matching name in a name chain, until we either find a sensible fit or the chain is exhausted.

With list construction. We construct the “with” list by adding a new node for every “with” list name. Now the “with” names are composite in general, but are incomplete. They may have a missing prefix (because we are in the scope of some covering “with” block) or a missing suffix (to be supplied by names in the block).

We therefore use the given strategy for each “with” list name, except that we work down into the type structure only as far as the name reaches. The last member of each name will be some selector; that selector must point to a type name, possibly through an array node, or else the name is in error. A pointer to that type name is then placed in the “with” list.

For example, consider the inner “with” name in the example:

```
with CONSWIDGET do    {creates the “*company” node}
begin
  .
  .
  .
  with HQTRS.STREET do {creates the
    “*streetaddress” node}
  begin
    .
    .
    .
  end;
end
```

The name “HQTRS” is a selector name, whose parent is “*COMPANY”. The “with” list contains one node for “*COMPANY”, indicating that the front end of this name is covered. We may then continue down the type structure, ending on the STREET selector node; this node points to “*STREETADDRESS”, which is what must be entered in the “with” list.

Exercises

1. Construct the type structure for the following set of Pascal types and variables:

```

type ALFA=array [1..10] of char;
    STATUS=set of (married, widowed, divorced, single);
    DATE=record MO:(jan, feb, mar, apr, may, jun, july,
                    aug, sept, oct, nov, dec);
                DAY: 1..31
                YEAR: integer
    end;
PERSON=record
    NAME: record FIRST, LAST:alfa end;
    SS: integer;
    SEX: (male, female);
    BIRTH: DATE;
    DEPDTS: integer;
end;
end;
```

2. Discuss implementation of a Pascal type structure and “with” list from productions. Could these be constructed in one stack?
3. Given the CONSWIDGET structure, show the “with” list at the inner block (→) of the following program segment:

```

with CONSWIDGET do
begin
    with OFFICERS[N], HQTRS.POBOX do
    begin
→
        end
    end
end
```

Give examples of legal names at this inner level.

4. Are ambiguous Pascal names possible? Give some reasonable definitions, and develop an algorithm for checking for ambiguous name references. Note that partially qualified names are not permitted and that a primitive data object must always be specified; Pascal contains no operations on a set of primitive objects.

8.4. String Tables and Their Access

A *string table* is some storage system that provides for the efficient insertion and retrieval of identifiers (the strings.)

We shall present four rather different search methods. These have applications other than in compilers, for example, data base systems and sort systems. Of the four, the *hash access* method seems to be the method of choice for compilers since it is demonstrably the most efficient for the table sizes encountered in typical programming languages.

These access methods have certain properties in common, illustrated in figure 8.10 for the linear method. Three stacks are maintained, a *scope* stack that keeps track of the identifier domains and a *pointer* stack that carries links to the *identifier* stack.

Two identical names in different scopes may be entered twice or once in the identifier stack, depending on whether one is interested in minimum identifier stack size or minimum access time. In order to share name space, it is necessary to search the entire name table, not just the current scope, on entering a new name. It is also necessary to maintain an additional pointer to the identifier table in the scope stack, in order to reduce the identifier stack correctly at the end of a block.

Identical names in different scopes are rare, hence it is probably not worth the additional search time to attempt to share names. We shall assume that names are not shared in the remaining discussion.

Upon entering a new block, a pointer table top-of-stack (TOS) index is pushed in the scope table. This action effectively “marks” the name table stack.

Upon leaving a block, the TOS scope table index is used to reduce the pointer stack, and the resulting pointer stack TOS index is used to reduce the name table. If the names from this scope are needed later (in a multipass compiler), they may be transferred to secondary storage before reducing the stacks.

Upon a declaration, a search is made in the current scope (as defined by the pointer stack TOS and the scope TOS index) for the name. If the search succeeds, we either have additional attributes for an existing name or a multiply-declared name. If the search fails, the name is pushed in the identifier stack and a pointer to it pushed in the pointer stack.

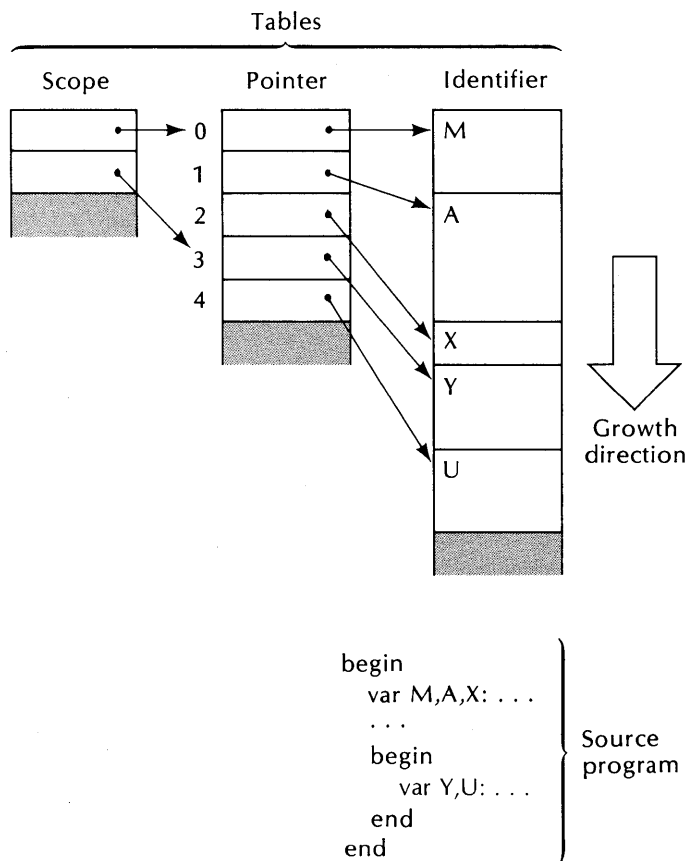


Figure 8.10. Linear name table-access method.

Upon a reference, a search is made in the entire table for the name, using the pointer stack indices. The search is conducted from the pointer stack TOS inward and stops on the first match or the bottom of the stack. If the name is not in the table, we may have an implicit declaration (as in Fortran) or an undeclared identifier. By stopping on the first match and by searching from TOS down, we locate that name most recently declared, hence the one declared in the innermost block.

The total declaration time t_D for a program will be different than the total reference time t_R for the following reasons:

1. The current scope is being built in the declarations, while it is essentially complete for references. (The truth of this assertion depends on the language. We are assuming Algol declaration rules for our discussion).

2. If the source program is free of declaration errors, an identifier will not be in the table for a declaration search, but will be in the table for a reference search.
3. A reference search may span the entire table, while a declaration search spans only the current scope. This search scope makes little difference in a language with only one block level, but makes considerable difference in a block-structured language. The difference is also influenced by the usage patterns of identifiers. The most heavily used identifiers are usually locals, and the next most heavily used identifiers are globals (block level 0).
4. A given identifier is declared once, but may be referenced several times in its block.

In order to make reasonably meaningful comparisons among the name access methods, we will develop “decl” and “ref” average times separately and make reasonable assumptions about the remaining factors. In section 8.4.5, the net access times of the four methods are compared. It is shown there that the hash access method is measurably superior to the others in access time. However, it requires somewhat more memory space than the others.

8.4.1. Linear Access

This method, the simplest of the four access methods, is illustrated in figure 8.10. On a declaration, a new name is simply pushed into the identifier table, and its index pushed in the pointer table. The names are clearly unordered, so that a search requires a comparison with each of the names in some region of the pointer table.

On a declaration, a search through the entire present scope is usually made. If the scope contains n names on the search, then the search time is

$$kn$$

where k is some unit comparison time. On each declaration, another name is added to the table, increasing n . If N' names are declared, then the net declaration time for one block is

$$t_D = \sum_{i=1}^{N'} ki = kN'(N' - 1)/2$$

The net reference time per block depends on the total number of active identifiers N in the table and on the usage pattern. Let us assume that references are distributed uniformly among the covering blocks and the current block. We know only that $N \geq N'$ and that each referenced identifier will be in the table (very nearly—barring programs with a large number of undeclared identifiers.) The average access time per identifier will be

$$kN/2$$

since we need scan only halfway through the table to find a given identifier (some will be near the top, some near the bottom of the identifier stack.)

We also need the average number of references per declaration b ; then given N' identifiers in the block, we have bN' references total, for a net reference time per block of

$$t_R = kbNN'/2$$

The block entry and block exit times are both small and independent of the number of identifiers.

If a sorted symbol table listing is required at the end of a block, the sort may be applied to the current scope of the pointer stack. A “shell sort” requires $n(\log_2 n)$ operations to sort n items, hence a net time

$$kN'(\log_2 N')$$

is required for the sort.

8.4.2. Binary Access

Suppose we are in the kitchen and wish to eat a cake, but learn that there is a needle in it. The cake knife is in the kitchen and can't be moved, and a metal detector is in the garage and mustn't be brought to the kitchen. The detector says only whether or not metal is in a piece of cake; it doesn't have the resolution to locate it.

An optimal approach is to cut the cake in half and take one half to the detector. It indicates whether the needle is in this half or the other. We eat the half without the needle and cut the remaining half in half, etc. On each trip to the garage, we eat another half of the remaining cake and can eventually eat all of it—the successive divisions will reduce the cake down to the needle itself. If the cake conceptually consists of N pieces the size of a needle, then we need only $(\log_2 N)$ divisions to complete the process. This method is obviously a sizable improvement over a linear search, which would require cutting the cake into N pieces and taking each piece to the metal detector.

A binary search for a new identifier starts at the middle of some region of the pointer table (initially the center of the table.) A comparison with the center identifier then indicates which of the two halves should be considered for the next region. If N identifiers exist in the table, then $(\log_2 N)$ comparisons are needed to locate an identifier.

The binary access method is illustrated in figure 8.11. Each new identifier is pushed in the identifier stack as usual. Identifiers are not shared, hence a given name may appear more than once in the identifier stack.

The pointer stack contains indices to the identifier stack as usual; however, it is arranged so that at all times, the names are sorted as seen through the pointer stack. Duplicate names carry adjacent pointers. The scope stack

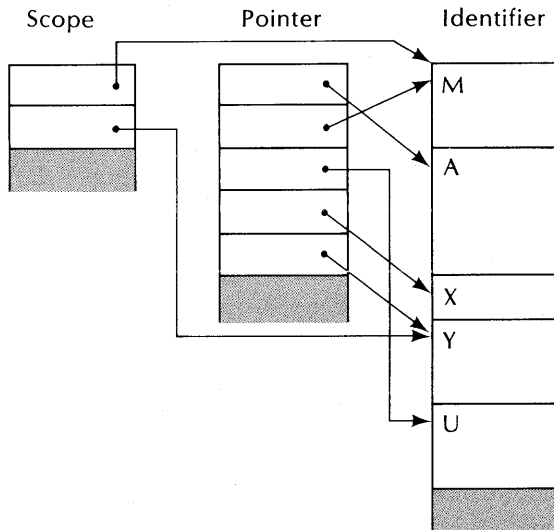


Figure 8.11. Binary name table-access method.

points to the identifier stack, not to the pointer stack; the latter does not have clearly delimited scope boundaries.

On a block entry, the current identifier TOS index is pushed in the scope table. The cost of this operation is small.

On a block exit, the pointer table must be purged of references to identifiers in the block being left, and then the pointer table must be compressed, which can be done efficiently in one scan of the pointer table. The scan moves pointers downward in the table, discarding those that belong in the current scope. Note that identifier comparisons (relatively expensive) are not involved in this operation.

A *declaration* requires a search followed by a sorted insertion. The insertion requires moving half the pointer table down, on the average, to make room for the new entry. Its location is clear from the search result. The move operation per item inserted is

$$a + k'n/2$$

where “a” is some move overhead time and k' is the time per unit move. Many computers have a fast move instruction that can carry out this operation, so that “a” and k' will be fairly small numbers.

The net time for a set of N declarations is then

$$\sum_{n=1}^N (a + k'n/2) = aN + k'N(N-1)/4$$

which must be added to the search and comparison times for the declarations, given by

$$\text{Search time} = k \left(\sum_{n=1}^N (\log_2(n) + 1) \right) \sim kN(\log_2(N) + 1)$$

On a reference, a search of the whole table must again be carried out. Now a reference match will be found before the search narrows down to one pointer. The net reference time is therefore approximately

$$t_R \sim kN'(\log_2(N'))/2$$

Although the reference time is proportional to the unit comparison time k , it increases much more slowly with N' than for the linear search method and will be smaller, unless blocks are typically small and global references infrequent.

The search algorithm should examine the neighborhood of a “found” identifier for other identical identifiers and choose that one in the most local scope. This selection can be made systematic by arranging that the most recently entered identifier have the greatest index of a set of identical ones, rather than just any appropriate index. Then on the search, there must be an added search toward larger indices to find the most recently entered identifier.

A sorted symbol table report obviously requires no sort upon a block exit. However, the pointer table must be completely scanned, and only those identifiers within the current block selected through the use of the scope table and the identifier table indices.

8.4.3. Tree Access

Figure 8.12 illustrates the tree access method. On a declaration, a new identifier is pushed into the identifier stack as usual. Then a new tree node is created, consisting of a “left” and a “right” pointer pair, along with an identifier pointer element. The left and right tables constitute a binary tree description. Each node either has no children, a left child, a right child, or both; ϵ indicates no child. In placing a new identifier in the tree, we compare the new one with the existing ones, then walk down the tree starting at the root. We place the new node in the first available child position. At each node, we move left if the new identifier is lower in collation order than the node and to the right otherwise.

For example, consider a new identifier B . Then $B < M$ so we move left and

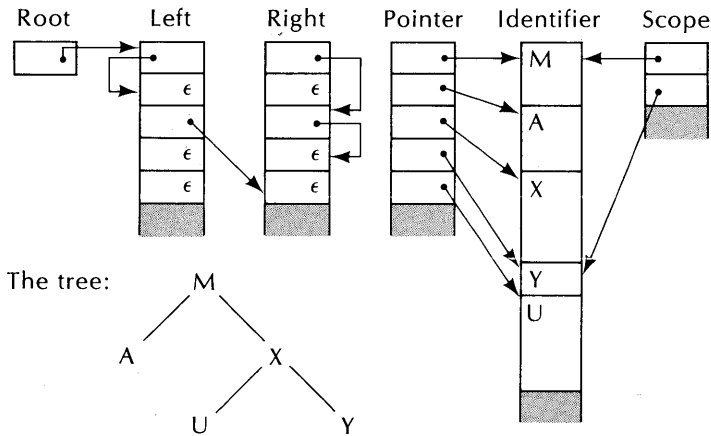


Figure 8.12. Binary tree name table-access method.

compare B with A. Now $B > A$, and A has no right child, so B goes into A's right child position. The new B entry has empty left and right children.

The average declaration time is $k(\log_2 N)$, plus a small time to add the four table entries. We do not have to move a list of pointers as we did with the binary method, so we appear to have a time improvement. However, k is somewhat larger for this time than for the binary search method, since a binary tree will never be completely balanced, which increases the average access time. In the binary search, the searching is always balanced.

The average use time is $k(\log_2 N)/2$; an identifier will be found halfway down the tree on the average.

The entry time is negligible; we merely add another item to the scope table.

The exit time is significant. We must walk through the tree and locate every node that points to an out-of-scope identifier (an identifier about to be dropped); this node must be changed to "empty." Then the identifier, left, right, and pointer stacks may be cut back.

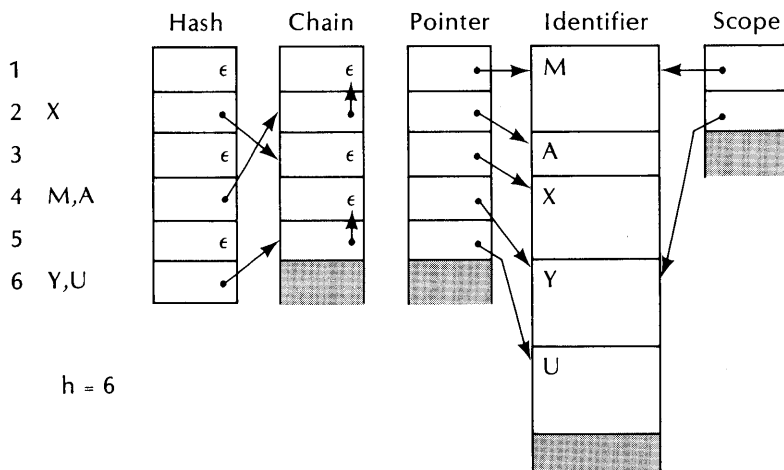
A sorted symbol table is generated by sorting a portion of the pointer table. Alternatively, a left-to-right tree walk yields a sorted list—those identifiers not in the current scope are ignored.

As in the binary search method, some rule must be established to distinguish identical identifiers in different scopes, e.g. always placed to the left of an existing one. Then in the search, given a match, the comparisons should continue down the left side until a mismatch is found; the last matching one is that most recently entered.

8.4.4. Hash Access

The hash access method is illustrated in figure 8.13. A *hash function* is defined on the class of identifiers; this function maps every identifier into an integer between 1 and h , where h is a fixed hash table size. Thus in the figure, X maps to 2, M and A map to 4, and Y and U map to 6. It is not essential that the hash function map every identifier to a distinct integer (indeed this is impossible if the number of identifiers is greater than h), but it should provide a reasonably random and uniform mapping. We call the hash function value for some identifier its *hash code*.

Given the hash code of an identifier, we enter the hash table directly through the hash code as an index and search for the identifier along a chain; if the identifier is in the table, it must be along that chain. We enter each identifier at the head of its hash code chain. Note that a declaration search need go down a chain only until the present scope is left. For a reference search, a chain must be followed to its end, if necessary, since an identifier may be in any scope.



The hash chains:

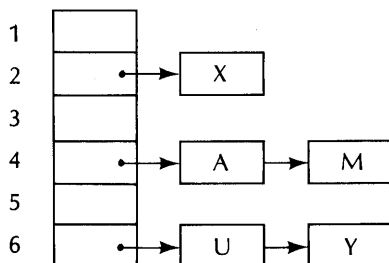


Figure 8.13. Hash name table-access method.

A *chain* table is shown in figure 8.13 to hold chain pointers; ϵ stands for the end of a chain.

For example, identifiers M and A hash to the same code, 4. Identifier M is entered first, by pushing it into the identifier stack. Corresponding entries are pushed in the pointer and chain stacks; the new chain index is 1, so the hash table entry is moved to the chain table (empty) and the hash table entry is set to 1. Eventually, the A is encountered. Repeating this process, we move the hash table entry to the new chain table entry (index 2) and place 2 in the hash table.

The configuration after all five identifiers have been entered is shown in the figure.

The declaration time for a name is kN/h . On the average, each hash chain contains N/h names from the current scope, and we must search one of them completely to establish that the name is missing.

The use time is $kN'/(2h)$, since on the average we hit the identifier halfway down a chain. Note that these times are essentially linear search times, reduced by a factor of h , the number of hash buckets.

Some overhead is incurred in computing the hash function, but it can be kept small by using a simple hash algorithm.

The entry time is negligible. The exit time is appreciable. We must search down all chains of the hash table entries, and relink the chains to the previous scope names. However, since the names are encountered last-in first-out, only the present scope name chain entries are scanned. Those deeper in the chains can be ignored.

The block exit time can be reduced through one of the following schemes:

1. Push a copy of the hash table into the scope stack on block entry, in addition to the TOS indices of the chain and pointer and identifier stacks. Then on exit, the hash table may be restored without a chain search.
2. Maintain a hash table stack, and start a new hash table, initially empty, on block entry. On block exit, it may simply be discarded. However, a reference search must then work through each of the stacked hash tables. No special time penalty is paid for this search, however, since each of the chains is smaller by the number of hash tables, on the average, and each of the tables is easily entered through the common hash code.
3. Add a set of pointers through the table entries that constitutes a linked list of entries that must be at the top of the hash chains upon block exit. This is a rather expensive approach, since a pointer cell is needed for each hash table cell and entry.

The names are not sorted in this scheme, so a sort algorithm must be applied to the pointer table to list the symbols alphabetically.

Bounded Table Hash Access

Sometimes we can guarantee that the number of symbols entered are less than or equal to h . If so, we don't need the overflow table; we can simply apply a *rehash* algorithm repeatedly until we find an empty cell in the hash table. Morris [1968] has shown that this algorithm is superior in performance to a hash chain method.

This method is useful for tables with a fixed number of entries, e.g., a table of reserved words for a compiler or a table of assembler mnemonics. However, it is usually unnecessary to maintain two distinct name tables—the reserved names can be accessed by the same algorithm as user identifiers. Their reserved status can be inferred from their position in the table.

An optimal strategy for such a symbol table, from Morris, is expressed by the following Pascal program:

```

var C,R: integer;
var IDENT: array [1..n] of char; {identifier}
var HASHTAB: array [0..h] of integer; {hash table}

procedure RAND: integer;
begin
  R:=(R*5)mod(h*4); {h is hash table size}
  return(R/4)
end;

{the hash access algorithm starts here}

C:=HASHIT(IDENT); {returns a hash code}

while HASHTAB(C)<>NULL do
begin {something in this hash chain}
  if NAMETAB(HASHTAB(C))=IDENT then return(TRUE);
    {found if match is found}
  C:=(C+RAND)mod(h); {try another hash code}
end;
return(FALSE) {not found}

```

The average number of probes required to locate an item known to be in the table is

$$E = 1 + (N/(h - N + 1)) \sim 1/(1 - a)$$

where h is the hash table size, N is the number of items in the table, and $a = N/h$ is called the *load factor*. Some sample values of E :

Load Factor a	E
0.1	1.11
0.5	2
0.75	4
0.90	10

A reasonable load factor is 0.75; 25 percent of the table is wasted, but an average of only four probes is needed to locate a name.

Hash Functions

We now consider the problem of a suitable hash function. We need a simple algorithm that will map a class of commonly used identifiers as uniformly as possible into an interval $1..h$, where the hash table contains h buckets. Let us consider a simple function, and evaluate its hash behavior:

$$C := \text{FIRST} + \text{LAST} + \text{LENGTH} - m$$

where **FIRST** and **LAST** are the numeric forms of the first and last characters of the identifier and **LENGTH** is its length. The parameter “ m ” is a constant, defined as follows.

Suppose the characters are ASCII 7-bit coded and consist of 1 to 15 letters and digits, starting with a letter. The ASCII codes are:

“A”=65, “B”=66, . . . “Z”=90, “0”=48, “1”=49, . . . “9”=57.

Now let $S = \text{FIRST} + \text{LAST} + \text{LENGTH}$. The least S is 115, for the identifier “A0” (A single letter identifier would have a larger S , since the letter is added in twice). The largest S is 195, for the identifier “XXXXXXXXXXXXXZ” (the X’s may be anything). We therefore set $m = 114$. The largest useful h is then $(195 - 114) = 81$. The class of identifiers can clearly fill the hash code interval $1..81$, so that no hash table cells will go completely unused.

An experiment with a program containing 623 identifiers and a hash table with 81 buckets yielded the following summary statistics:

Chain Length L	Number of Chains with Length L
0	24
0/4	41
5/9	9
10/14	9
15/19	12
20/24	6
25/27	3
>27	0

The net cost of accessing the 623 identifiers through the hash chain system was 9417 name comparisons. Now the average chain length is 7.7 names ($623/81$), so that the 5 to 9 chain length range should predominate. Instead, we find that almost half the chains carry less than 5 names, and several chains carry more than 25 names. Clearly, something is wrong with this hash function.

We clearly cannot leave an identifier hash function to chance. We may end up with many of the cells unused or a group of cells much more heavily used than others.

Knuth [1973] showed that an effective hash function on some key K is simply a modulo h division or multiplication. The key K must be some integer, possibly formed from a few characters in the identifier by concatenation. Then

$$h(K) = K \bmod M$$

where M should be a prime number yields a reasonably random hash code. Values of M which divide $r^k + a$, or $r^k - a$, where k and a are small numbers and r is the radix of the alphabetic character set (usually 64, 256, or 100, depending on the machine), should be avoided.

Suppose we have 32-bit integer arithmetic. Then we can form K from the identifier string $a_0 a_1 \dots a_n$ by the following algorithm:

```
var D: double integer;
if n > 4 then D := [a0, a1, an-1, an]
    {formed by concatenating the
     first two and the last two characters}
else D := [blank, blank, . . ., a0, . . ., an]
    {left-most blank fill}
```

The following table shows the results of an experiment with this same program, containing 623 identifiers but using a Knuth hash function:

Chain Length L	Number of Chains with Length L
0	0
0/4	9
5/9	54
10/14	17
15/19	1
>19	0

Clearly, the chain lengths are more uniform. The net access cost is 5,794, about half that for the hash function above. The dominant chain length is closer to the expected average, 7.7, and there are no unused or unusually large chains.

8.4.5. Comparison of Access Methods

The four access methods described may be compared through their required space or time. In comparing space, we note that all four require the same identifier, scope, and pointer space. We therefore compare the space required in addition to this:

- Linear: 0
- Binary: 0
- Tree: $2*N$, where N =number of table symbols
- Hash: $h + N$

A comparison of access times is somewhat more difficult. We have previously developed formulas for entry, exit, declaration and reference (use) times. These formulas are summarized in the next table.

Summary of Access Times

	Entry	Exit	Declaration	Use
Linear	10	10	$25*N'*(N'-1)$	$25*N*N$
Binary	10	$16*N$	$25*N*(\log(N)-1)$	$25*N*\log(N)$ $+ 5*N + k'*N*(N-1)/2$
Tree	10	$8*N$	$35*N*(\log(N)-1)$	$35*N*\log(N)$
Hash	10	$8*N'$	$25*N'*(N'-1)/h$	$25*N*N/h$

Each of the table entries represents some net access time, in arbitrary units, for a typical program block. Here N is the number of identifier declarations in the block, and N' the number of identifier references. Parameter h is the number of hash buckets. The remaining factors have all been estimated, based on our experience. For example, the use time for tree access is similar to that for a binary sort, except that the factor (35) is larger because of tree imbalance. The binary sort exit time is larger than a tree exit time on account of the pointer table compression required.

We next fold these time estimates into the following set of frequencies, given by McKeeman [1974b]. These apparently are based on experience with a number of XPL (block-structured) programs:

f1, entry	f2, exit	f3, decl	f4, use
10	10	100	700

We then may compute an average unit access time, through the following averaging formula:

$$\text{Time} = (f1 \cdot \text{entry} + f2 \cdot \text{exit} + f3 \cdot \text{decl} + f4 \cdot \text{use}) / (f1 + f2 + f3 + f4)$$

We assume that $N' = N/5$, i.e., for every identifier declared in the current scope, there are four others in covering scopes in the symbol table. (This number 5 may be too large for an average program, but it doesn't affect our conclusions much.)

We also assume that the number of hash buckets h is 100.

The graph in figure 8.14 shows the net time as a function of N in the range 1 through 360. The hash table must be nearly full at $N=360$, yet the hash method remains superior to the others far beyond this N . In fact, the hash and binary methods have about the same time at $N=1200$: binary is 288, hash is

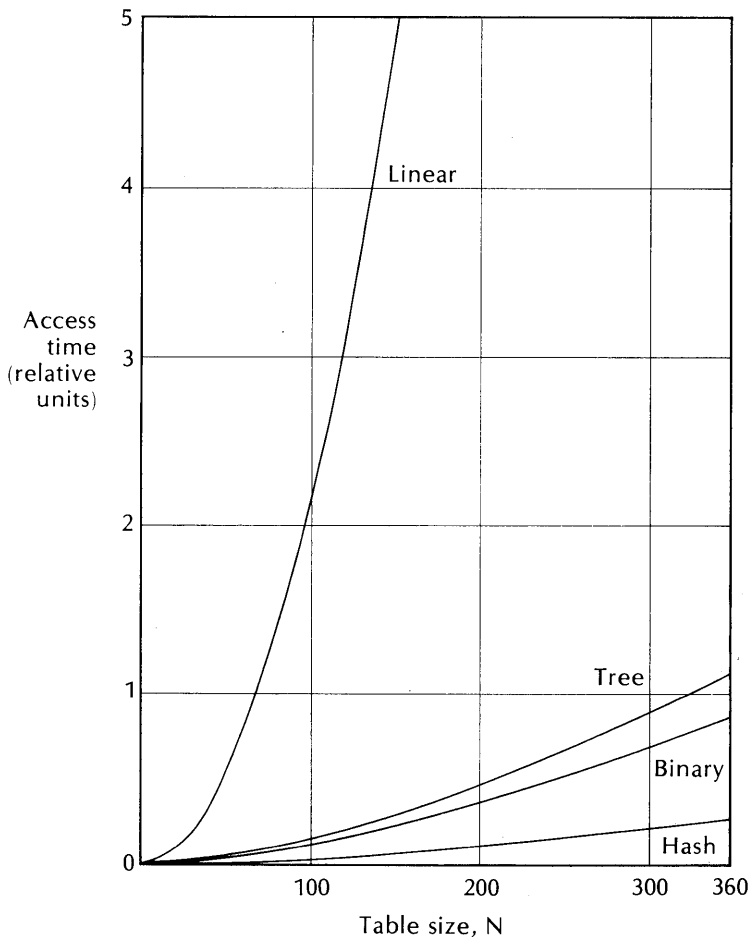


Figure 8.14. Comparative access times for the four access methods, linear, tree, binary and hash, as a function of table size.

279. At $N = 1200$, the average hash chain is 12 symbols long, requiring about 12 comparisons. However, 1200 is approximately 2^{10} , so the binary method also requires about 11 comparisons.

For $N > 1200$, the binary method is the most efficient; however, the efficiency of the hash method increases with the number of hash buckets h , so that a larger h might be justified for such a large string table.

The tree method is consistently poorer than the binary method; note that it also requires more storage. The linear access is orders of magnitude more costly in time than any of the other three. At only 100 symbols, linear access requires 100 times the search time as hash access. (This factor of 100 is the number of hash buckets.)

Summary of access methods

The most efficient access method, by time comparison, is the hash access method. It requires a function that maps an identifier into a finite range of integers 1 to h in a uniform manner. The hash code resulting from the mapping is then used to isolate the identifier to one of h separate linear chains of identifiers. The desired identifier is found by a linear search along that chain. Although the binary and tree search methods result in a sorted table, convenient for a symbol table listing, this apparent advantage is outweighed by the larger overhead in declaration and reference times.

8.5. Bibliographical Notes

The use of symbol tables in computer systems appears to have ample precedent. access methods, with detailed examples, are given by McKeeman [1974b]. Knuth [1968], vol. 1, chapter 2, contains a treatment of symbol table tree structures suitable for PL/I or Cobol names. Knuth [1973] discusses searching and hashing at length.

A formal symbol table and attribute system of considerable power and generality, called *property grammars* have been recently defined by Stearns and Lewis (Stearns [1969]). A good review of property grammars, with examples, is given in Aho [1972a], vol. 2, chapter 10; Aho also develops statistical formulas for the evaluation of the efficiency of the hash table search method.

RUN-TIME MACHINE STRUCTURES

9.1. Introduction

In the last chapter, we developed symbol table structures that the compiler should maintain as a means of keeping track of (1) the attributes of the various data objects that will appear in code at run-time and (2) information related to the blocks and procedures of the source program. We now need to describe run-time structures which entails the allocation of variable space and mechanisms for dealing with procedure calls, parameter passing, and variable access.

We can develop a system of run-time structures without resorting to coding details of any particular machine by developing a minimal set of instructions for a fictitious machine that supports the data structure and procedure mechanisms required for Algol 60, Pascal, and PL/I. Embedded in these instructions is a subset that can support Fortran or Basic. Given an Algol-like language, those machine features that suit that language can be selected from this machine and perhaps simplified or modified to suit the desired language properties.

In each of the above languages the dimensions of vector or matrix data objects may be dynamically altered, though in rather different ways in the different languages.

A more general dynamic data allocation system is required in languages such as APL and Lisp, where data may be allocated and restructured “on the fly” as it appears in various executable expressions. Data objects are never declared as such in APL; their characteristics are inferred from operations that set their values. We shall not discuss data allocation for APL and Lisp, although certain of the ideas expressed in this chapter are applicable to interpreters of these languages.

The fictitious machine instructions developed in this chapter are sufficiently well defined that a simulation program can be written in any common programming language. We suggest that such a simulator be written and used to study the machine operations developed in this chapter.

9.2. Run-Time Structures for Algol-Like Languages

We now develop a run-time stack machine system that can support essentially all of the features of Algol 60, Pascal, and PL/I.

This machine, called the AOC machine (for Algol Object Code), we owe to Randell and Russell (Randell [1964]), with revisions by James Morris and the authors. Its mechanism focuses on the important problems of variable scope conventions, nested procedure declarations, allocation and assignment of memory space to variables, access of variables, indexed variable conventions, and procedure call conventions.

The AOC machine has a linear sequential data memory with locations addressed by $h, h+1, \dots, -1, 0, 1, \dots, s-1, s$, each of which can hold a numeric value (an integer or a real) or a truth value (TRUE or FALSE). The portion of data memory accessed by negative locations is called the *heap*, and the remaining portion is called the *stack*. The data memory is regarded as an infinite two-way tape; the locations may extend arbitrarily far in either the positive or negative direction during execution of a program. The largest positive location in use during execution is denoted by s and the most negative location by h . Both the heap and the stack are initially empty, hence initially $h=0$ and $s=-1$.

The AOC machine also contains a finite linear sequential program memory with locations addressed by $0, 1, 2, \dots, n$. The contents of this memory and its size cannot be changed during execution. The program memory is initially loaded with program instructions and thereupon remains fixed until the end of the program. Location 0 is the first executable instruction. No data is located in the program memory. Although it could be used to hold constant data, difficulties in accessing such data through an address label would thereby be created.

We shall not be concerned about the number of bits per word, or possible packing of multiple elements per word. We shall also not be concerned about allocation and assignment of physical memory to the data and program memories. We assume that sufficient memory space is always available to support whatever program or operations are required of the AOC machine.

The contents of cell j in data memory is denoted by $C(j)$. The notation

$$C(j) \leftarrow x$$

means "store the value x in location j ." The notation

$$C(j) \leftarrow a, b, c$$

is equivalent to the set of commands

$$C(j) \leftarrow a; j \leftarrow j+1;$$

$$C(j) \leftarrow b; j \leftarrow j+1;$$

$$C(j) \leftarrow c; j \leftarrow j+1;$$

i.e., the set a, b, c are stored in consecutive locations in memory. Then the notation

$$x, y, z \leftarrow C(j)$$

is equivalent to

$$z \leftarrow C(j); j \leftarrow j - 1;$$

$$y \leftarrow C(j); j \leftarrow j - 1;$$

$$x \leftarrow C(j); j \leftarrow j - 1;$$

so that when the list (a, b, c) is stored and later restored in (x, y, z) , x corresponds to a , y to b , etc.

The contents of cell i in program memory is denoted $PB(i)$. Since program memory is read-only, only operations of the form

$$x \leftarrow PB(i)$$

are legal. In fact, the function $PB(i)$ will only be required in the master execution procedure, described later.

If $i < 0$ or $i > n$ then $PB(i) = \text{"HALT"}$. The master execution program may access $PB(n+1)$, but no instruction outside the range $0 \leq i \leq n$ should ever be executed in a valid program.

The program memory is fixed at compile time, in principle at least. An implementation may defer some operations on the program memory to just prior to execution of the program. However, no alterations in program memory are permitted during execution.

There is also another small independent memory called the *display*. Its cells are numbered $0, 1, 2, \dots, b$ and always holds certain addresses of the data memory. Here, " b " is the largest display index, and will vary during execution. The display is designated D , e.g.,

$$D(3) \leftarrow 7$$

$$C(5) \leftarrow D(6) + 2$$

are possible operations. The display will track the current depth of nesting in nested procedures or blocks, so that the maximum b is the largest depth of nesting found in a source program. Wirth found that $b_{\text{MAX}} = 3$ was sufficient for a Pascal compiler (Wirth [1971a]), so that the display size can be quite small indeed. We shall also see that a copy of the display must be kept in the data memory, and is always accessible, so that as it turns out, the display is needed only for the sake of efficiency and clarity.

The machine also has four special registers for the variables i , s , h , and b .

The overall operation of the AOC machine is described by the master execution program below. Initially, data memory is empty, $b = s = -1$, and $h = i = 0$. The program memory is loaded with instructions, and then

execution of the instructions begins at location 0. The program will expand and contract the data memory and display through its instructions and will stop upon encountering the HALT instruction.

```
{AOC machine master execution program}
var INS: integer;
LOAD(program);
s:=-1;
h:=0;
b:=-1;
i:=0;
INS:=PB(i);  {first instruction}
while INS ≠ 'HALT' do
begin
  i:=i+1;
  EXECUTE(INS);  {execute the instruction}
                {Note: Execution of INS may change i}
  INS:=PB(i)    {fetch the next instruction}
end;
```

The description of the individual instructions and their rationale constitute the rest of this chapter. The memory and register structures of the AOC machine are shown in figure 9.1.

A complete list of the instructions and their effect is given in section 9.9; they represent an instruction set for full Algol support. We shall evolve toward that set by starting with fairly simple definitions, then expanding them as the need arises.

We shall see that these are powerful instructions, despite their apparent simplicity. They are also easily implemented in any of several different ways, namely:

1. Build a machine according to the specification of these notes, perhaps through a combination of hard wired logic and microprogramming.
2. Write an interpreter for the instructions. This interpreter might be a program that executes Snobol instructions on a CDC 6400, or an assembly language interpreter implemented on a microprocessor.
3. Write a program to translate AOC into machine code for some machine.
4. Write a set of macros that generate machine code in assembly language.

9.2.1. Arithmetic and Logical Expressions

An expression like

$$A + B/7 - 3*D$$

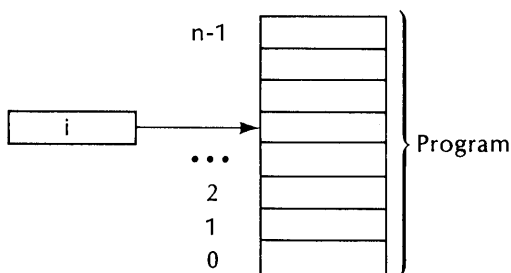
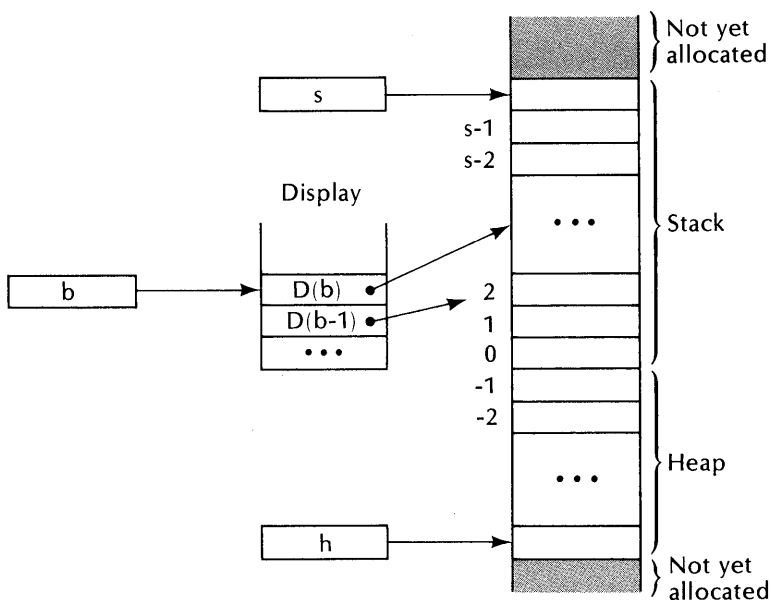


Figure 9.1. Memory and registers of the Algol object code (AOC) machine.

can easily be evaluated by a stack machine, as we have seen in chapter 7. A bottom-up or top-down compiler can simply emit stack load instructions on each variable and appropriate stack operations on each operator, as they are encountered in the parsing process. We are thus led to the following AOC arithmetic instructions:

LC n : Load constant n
 $s \leftarrow s + 1$
 $C(s) \leftarrow n$ { s marks the stack top}

LV a: Load value contained in address a

$$s \leftarrow s+1$$

$$C(s) \leftarrow C(a)$$

(Note: For the moment, an address is simply an integer. Eventually, an address will become more complicated.)

ADD: Arithmetic addition

$$C(s-1) \leftarrow C(s-1) + C(s)$$

$$s \leftarrow s-1$$

SUB: Arithmetic subtraction

$$C(s-1) \leftarrow C(s-1) - C(s)$$

$$s \leftarrow s-1$$

MULT: Arithmetic multiplication

$$C(s-1) \leftarrow C(s-1) * C(s)$$

$$s \leftarrow s-1$$

DIV: Arithmetic division

$$C(s-1) \leftarrow C(s-1) / C(s)$$

$$s \leftarrow s-1$$

Assume that A, B, and D in the expression $A + B/7 - 3*D$ are stored in locations -102, 120, and 200, respectively, and that $s = 300$. Then the following instruction sequence will compute the expression's value, leave it in location 301 and leave $s = 301$:

LV -102 {A}

LV 120 {B}

LC 7

DIV

ADD

LC 3

LV 200 {D}

MULT

SUB

The following instructions provide more arithmetic and logical capability:

NEG: Negate

$$C(s) \leftarrow -C(s)$$

NOT: Logical complement

(defined only for $C(s) = \text{TRUE}$ or FALSE)

$$C(s) \leftarrow \begin{array}{ll} \text{if } C(s) = \text{TRUE} & \text{then FALSE} \\ & \text{else TRUE} \end{array}$$

EQ: Equal
 $C(s-1) \leftarrow \text{if } C(s-1)=C(s) \text{ then TRUE}$
 else FALSE
 $s \leftarrow s-1$

EQ compares the top two stack values as integers and replaces them by a single Boolean value.

LS: Less then
 $C(s-1) \leftarrow \text{if } C(s-1) < C(s) \text{ then TRUE}$
 else FALSE
 $s \leftarrow s-1$

AND: Logical and
 $C(s-1) \leftarrow \text{if } C(s-1)=\text{TRUE and } C(s)=\text{TRUE}$
 then TRUE else FALSE
 $s \leftarrow s-1$

The AND operation compares the top two stack values as Boolean variables and replaces them with a TRUE or FALSE, according to the usual meaning of logical AND. The operations AND and OR are only defined if the top two stack elements are in the set {TRUE, FALSE}.

OR: Logical or
 $C(s-1) \leftarrow \text{if } C(s-1)=\text{TRUE or } C(s)=\text{TRUE}$
 then TRUE else FALSE
 $s \leftarrow s-1$

9.2.2. Assignment Statements

The statement

$$A := B + 1$$

is supposed to calculate the value of $B + 1$, then set the cell assigned to A with the result. We introduce the instruction STD to accomplish this end:

STD a: Store direct in a
 $C(a) \leftarrow C(s)$
 $s \leftarrow s-1$

Then the statement $A := B + 1$ is executed by the instruction sequence

LV B
 LC 1
 ADD
 STD A •

We also must support access of indexed variables, e.g., $A[I]$, where the address of $A[I]$ is $(\text{address of } A) + C(I)$. Recall that instructions may not be changed, so we cannot just change a STD A during execution of the program. We need instead a set of instructions LA, ST, and CONT as follows:

LA a: Load Address a
 $s \leftarrow s + 1$
 $C(s) \leftarrow a$

This appears to be just like LC. However, as we add language features, addresses will get more complicated and LA and LC will differ.

ST: Store
 $C(C(s-1)) \leftarrow C(s)$
 $s \leftarrow s - 2$

ST expects an address in $C(s-1)$. It stores $C(s)$ into that address.

CONT: indirection
 $C(s) \leftarrow C(C(s))$

CONT expects an address on the stack top and replaces it with the value in that address.

An assignment statement of the form

$$A[I] := B[J] + 1$$

can then be coded as:

```

LA    A    {Load address of A, for the sake of
            the ST later}
LV    I
ADD   {address of A[I] now on stack}
LA    B
LV    J
ADD   {address of B[J] now on stack}
CONT  {value of B[J] now on stack}
LC    1
ADD   {B[J] + 1 value is in C(s),
      address of A[I] is in C(s-1)}
ST
```

Exercise

Write AOC code for the following statements:

```

A:=B*C - D*E + 75
B:=F AND (D OR E)
X[I+15]:=-Y/X[I-5]

```

9.2.3. Conditionals

There are several kinds of conditional constructs in the Algol-like languages:

1. Conditional expressions such as

if $A=0$ then 1 else $1/A$

2. “One-armed” conditional statements such as

if B then $B := \text{FALSE}$;

3. “Two-armed” conditional statements such as

if B then $A := 1$ else $D := 1$;

4. Loop conditional statements such as

while $A < 15$ do $A := A + 1$;

These and other conditionals may be handled by the following two instructions:

```

JP  1:  Jump to location 1
      i ← 1

```

Recall that i is the instruction register; it contains the location of the next instruction to be executed.

```

JIF 1:  Jump to 1 if top of stack is FALSE
      if  $C(s) = \text{FALSE}$  then  $i \leftarrow 1$ 
       $s \leftarrow s - 1$ 

```

Note that we “fall into” the instruction following the JIF if the top of stack contains TRUE. In any case, the Boolean value on the stack top is removed.

Let us now translate the Algol control statements given above into AOC code sequences.

1. The expression

if $A=0$ then 1 else $1/A$

is translated as:

```

    LV  A
    LC  0
    EQ   {TRUE or FALSE is left in C(s)}
    JIF  L1
    LC   1 {The THEN part}
    JP   L2
L1  LC   1
    LV  A {The ELSE part}
    DIV
L2  . . .

```

We use symbolic names L1 and L2 to denote locations to simplify the discussion. When the program is loaded, of course, the L1 in the “JIF L1” must be set to the location of the “LC 1” instruction when it is found later on.

2. The statement

if B then B:=FALSE

is translated as:

```

    LV  B
    JIF  L1
    LC  FALSE
    STD B
L1  . . .

```

3. The statement

if B then A:=1 else D:=1

becomes:

```

    LV  B
    JIF  L1
    LC   1
    STD A
    JP   L2
L1  LC   1
    STD D
L2  . . .

```

4. Finally, the loop statement

while $A < 15$ do $A := A + 1$

becomes

```

L1  LV  A
    LC  15
    SUB
    NOT
    JIF  L2
    LA  A
    LV  A
    LC  1
    ADD
    ST
    JP   L1
L2  . . .

```

Of course, we can also translate simple GOTO statements using the JP instruction. For example, the nonterminating Algol program

```

M:  A:=A+1;
    goto M;

```

may be translated to

```

L   LA  A
    LV  A
    LC  1
    ADD
    ST
    JP  L

```

9.3. Stack and Heap Allocation

Most of the Algol-like languages provide for dynamic allocation of memory. The allocation can occur in one of two ways as follows:

1. Space is allocated upon entering a block and encountering a declaration. The upper and lower limits of an array declaration may depend on run-time variables. The allocated space may not be changed within the scope of the block and vanishes upon leaving the block.
2. Space is allocated upon executing some run-time function or is the result of a declaration (with new dimensions) of an existing OWN array.

Space for arrays of type (1) can be allocated from the stack. Space for arrays of type (2) require the heap and a memory manager, as we shall explain in greater detail later.

For now, we introduce three instructions that facilitate the dynamic allocation of stack and heap space:

INCS: Increment stack pointer ($C(s) \geq 0$)
 $C(s + C(s)) \leftarrow s$;
 $s \leftarrow s + C(s)$

This expects a positive count N on the stack top. It allocates N words, and leaves the address of the allocated space on the stack top. The address may then be deposited somewhere, to form an array base address.

INCS n : Increment stack pointer ($n \geq 0$)
 $s \leftarrow s + n + 1$

This form of INCS simply allocates a fixed number of words, n . No address is left on the stack top. Since n is known to the compiler, the addressing operations related to this INCS are compiler operations. The two forms of INCS could be combined through a rule that if $n=0$ in INCS n , the stack count is the top value on the stack.

DECH: Decrement heap pointer
 $h \leftarrow h - C(s)$
 $C(s) \leftarrow h$

This modifies the heap memory extent (in either direction) by the number of words found on the stack top. The new h is returned and replaces the count word. This would be the address of an allocated heap array if $C(s) > 0$ initially.

DECH n : Decrement heap by n words
 $h \leftarrow h - n$
 $s \leftarrow s + 1$
 $C(s) \leftarrow h$

This decrements the heap pointer by n words, and pushes the current heap bottom on the stack.

DECS: Decrement stack ($C(s) \geq 0$)
 $s \leftarrow s - C(s) - 1$

This decrements the stack pointer by the number of words left on the stack top.

DECS n : Decrement stack ($n \geq 0$)
 $s \leftarrow s - n$

This decrements the stack pointer by n words.

9.4. Input-Output

The Algol 60 report contains no I/O definitions. However, we may easily add two primitive I/O operations. These are similar to those defined in Pascal:

- READ: A parameterless procedure which returns an integer read from some input tape as its value and also causes the tape read head to advance one position.
- PRINT: The value on the top of the stack is printed on an output tape or other medium, then removed from the stack top.

We conceive of input as a sequence of numbers on some input tape, and output as an endless tape, written one number at a time. Very little imagination is required to change the printing and reading mechanisms to accept characters, which, after all, can be considered special number codes. Then

PRINT(READ)

prints the next input number on the output medium.

The AOC instructions for I/O are

READ: Read number from input
 $s \leftarrow s + 1$
 $C(s) \leftarrow$ the next number on the tape
 (Tape input pointer moved one position.)

PRINT: Print top of stack onto tape
 Print $C(s)$ on the output medium
 $s \leftarrow s - 1$
 (Tape pointer moved one position.)

We need one last instruction (for now), HALT, which simply stops the computer.

9.5. Blocks and Storage Allocation

We have not yet discussed allocation of variables to the memory. The easiest plan is simply to allocate variables in stack addresses 0, 1, 2,

This simple plan effectively gives us a Fortran machine, since in Fortran, each variable is given a fixed, permanent location in memory. However, in Algol, it is possible to declare some variables in the middle of a program, use them for awhile, then let them “go away”; such variables are said to have a limited scope (the region between their declaration and “letting them go”), and may not be accessible outside their scope. The advantage of the Algol system is two-fold: memory may be reused for different local variables, and the same name may be used in different scopes without conflict.

For example, the following Algol program reads three numbers and prints the first two in the same order as read, or the reverse order, depending on the sign of the third number.

```
begin integer A, B; {start of scope 1}
  A := READ;
  B := READ;
  if READ < 0 then
    begin integer T; {start of scope 2}
      T := A;
      A := B;
      B := T;
    end; {end of scope 2}
  PRINT(A);
  PRINT(B);
end {end of scope 1}
```

The variable T is valid only within the scope 2; variables A and B are valid anywhere within scope 1. Variable T is only needed as a temporary for the interchange of A and B.

The range of use of variables is governed by the Algol *renaming rule*:

Algol Renaming Rule (TRR). If all the occurrences of an identifier declared at the beginning of a block are replaced by a new identifier which does not occur anywhere within the block, the meaning of the program does not change.

Note that the occurrence of the identifier in the declaration must be changed as well. For example, the program

```
begin integer X;
  X := 1;
  begin integer X;
    X := 2;
  end;
  PRINT(X);
end
```

is equivalent, by TRR, to

```
begin integer X;
  X := 1;
  begin integer Y;
    Y := 2;
  end;
  PRINT(X);
end
```

The point of TRR is that it leaves no doubt that this program prints “1”, not “2”. Another way of looking at TRR is that variable X in the first program statement is “suspended” by the inner block declaration of X; its value is picked up again at the end of the inner block. TRR clearly demands two different cells to be assigned to the two variables X and Y in the second program statement.

Now let us sketch a Fortran-style allocation system. To translate a program:

1. Use TRR so that every identifier has exactly one declaration. Let there be m different identifiers after this is done.
2. Translate the program (putting the first instruction in location 0) using the identifiers for addresses. Let the program translate into n AOC instructions.
3. Associate the m identifiers with locations $0, 1, 2, \dots, m-1$ in data memory. Then replace the identifiers in the instructions with their associated locations.
4. The first program instruction will be:

INCS m

to allocate m words for the variables.

The translation of the reversal program (above) by this procedure would then be:

```
0 INCS 3 {space for the three variables}
1 LA 0 {variable A}
2 READ
3 ST
4 LA 1 {variable B}
5 READ
6 ST
7 READ
```

```

8  LC  0
9  LS
10 JIF 20
11 LA  2  {variable T}
12 LV  0  {variable A}
13 ST
14 LA  0  {variable A}
15 LV  1  {variable B}
16 ST
17 LA  1
18 LV  2
19 ST
20 LV  0
21 PRINT
22 LV  1
23 PRINT
24 HALT

```

If we wanted to conserve data space, we could permit some of the variables to share the same space. Two variables can share the same data memory space if they are never simultaneously active and are both local. (Fortran variables cannot be shared.) Thus consider

```

begin integer x;
.
.
.
begin integer a,b,c;
.
.
.
end;
.
.
.
begin integer d,e,f,g;
.
.
.
end;
.
.
.
end

```

The two sets of variables $\{a,b,c\}$ and $\{d,e,f,g\}$ are never active at the same time. They are also undefined upon entry to a block, so three of them could share the same storage locations. Then the above program would require only five storage locations:

- One for x in the outer block.
- Four for $\{x,a,b,c\}$ in the first inner block.
- Five for $\{x,d,e,f,g\}$ in the second inner block.

This Fortran-style scheme breaks down if recursive procedures are permitted in the language.

Exercises

1. Translate the following Algol program segment into an AOC instruction sequence:

```
integer I, J, K, L, M;
J:=15;
I:=0;
M:=3;
while I<READ do
begin
  M:=M+1;
  J:=J+I;
  if J>20 then J:=0
end
```

2. Write an AOC code sequence that will allocate space for a one-dimensional array (size n words) and fill it with zeroes. Consider two cases: (a) where n is a compile-time constant, and (b) where n is obtained by first evaluating some expression at run-time.
3. Suppose that several one-dimensional arrays are wanted, each of whose dimensions are evaluated during execution upon entering some block. How would they be allocated in the stack? Is it possible to write access instructions for them in AOC code, as developed so far? How might the array elements be accessed? Recall that the instructions cannot be altered at run-time.
4. Write an AOC program that reads a list of numbers terminated by -1 , then counts and reports the number of 0's, 1's, 2's, ..., n 's found. (Here n is the first number in the list, and the list contains only nonnegative integers, except for the terminating -1 .)

5. Sketch an algorithm that allocates data space for variables and fixed-dimension arrays for a block-structured Algol 60 program, with maximum data space sharing.
6. Develop AOC code sequences to support a REPEAT UNTIL structure, as in Pascal.
7. Develop AOC code sequences to support a Pascal labeled CASE statement. Are any new instructions needed? If so, define them.

9.6. Procedures and Recursion

One or more procedures in Algol may be declared at the beginning of any block. A procedure declaration consists of two parts, a *heading* and a *body*. Its heading will be (for now) simply the key word **PROCEDURE** followed by its name and a semicolon. Its body may be any statement and most often is a block. Like other variables, a procedure name is known and may be called anywhere within the block in which it is declared; it is unknown outside that block. It is also known within its own block, hence may call itself.

Procedure semantics are governed by *the procedure copy rule*:

Procedure Copy Rule (TPCR). The effect of a procedure call shall be as if the body of the procedure were to replace the calling statement, then be executed. (Some complications will be brought out later.)

We may regard the copying as being done while the program is being executed by a machine which executes Algol programs directly, with no translation. For this simple case, the copying could be done at any time, even before the program starts.

Consider the following program, which prints the numbers on its input tape in reverse order:

```
begin procedure REV;    {procedure body beginning}
  begin integer X;
    X := READ;
    if ~(X=0) then REV;
    PRINT(X)
  end;    {end of the procedure body}
REV;    {The procedure call}
end
```

Let us trace the execution of this program in a symbolic way. We shall do so by simply writing out the program steps that have been executed, and placing an arrow before the statement that is about to be executed. A variable may be defined, indicated: "X=6", or may be undefined, indicated: "X=?".

The first executable statement in the reverse program above is the call REV. Let the input tape contain the numbers 5, 6, 0. Then shortly after the first call, we have

```
begin procedure REV;
  begin integer X;
    X := READ;
    if  $\sim(X=0)$  then REV;
    PRINT(X)
  end;
  begin {Copy of procedure REV}
    integer X1=5;
    X1 := READ;
    → if  $\sim(X1=0)$  then REV;
```

It is clear that we need another copy of REV at the point of call; TPCR and TRR must be applied again, yielding:

```
begin procedure REV;
  begin integer X;
    X := READ;
    if  $\sim(X=0)$  then REV;
    PRINT(X)
  end;
  begin {Copy of procedure REV}
    integer X1=6;
    X1 := READ;
    if  $\sim(X1=0)$  then
      begin {Second copy of REV}
        integer X2=6;
        X2 := READ;
        → if  $\sim(X2=0)$  then REV;
```

One more copy will do the trick. Note that we have introduced two variables, X1 and X2, and are about to introduce a third variable X3.

```
begin procedure REV;
  begin integer X;
    X := READ;
    if  $\sim(X=0)$  then REV;
    PRINT(X)
  end;
```

```

begin {Copy of procedure REV}
  integer X1=5;
  X1 := READ;
  if  $\sim(X1=0)$  then

    begin {Second copy of REV}
      integer X2=6;
      X2 := READ;
      if  $\sim(X2=0)$  then

        begin {Third copy of REV}
          integer X3=0;
          X3 := READ;
          if  $\sim(X3=0)$  then REV;
        →

```

The zero returned by READ this time indicates an end of tape. The “if” test on $\sim(X3=0)$ fails, so another copy is not needed. We can then unravel the recursive calls, by writing in the remaining statements of the procedure REV:

```

begin procedure REV;
  begin integer X;
    X := READ;
    if  $\sim(X=0)$  then REV;
    PRINT(X)
  end;
  begin integer X1 = 5; {Copy of REV}
    X1 := READ;
    if  $\sim(X1=0)$  then
      begin {Second copy of REV}
        integer X2=6;
        X2 := READ;
        if  $\sim(X2=0)$  then
          begin integer X3=0; {Third copy}
            X3 := READ;
            if  $\sim(X3=0)$  then REV;
            PRINT(X3) {"0" is printed}
          end;
          PRINT(X2) {"6" is printed}
        end;
        PRINT(X1) {"5" is printed}
      end;
    end;
  end
end

```

Notice that REV contains an error; we really don't want to print the end-of-list indicator 0. The procedure should read:

```

procedure REV;
begin integer X;
      X := READ;
      if  $\sim(X=0)$  then
        begin REV;
          PRINT(X)
        end
      end
end

```

The point of all this tracing is that a recursive procedure may generate many more distinct variables, or rather, activations of variables, than appear in the source program. Thus X1, X2, X3, ... must be treated as distinct and therefore require separate storage. If the same storage location were used for each activation, the program would simply print a series of $N+1$ zeroes instead of the intended reversed sequence.

The AOC machine must therefore be prepared to supply an arbitrarily large amount of storage for a running program to accommodate recursive procedures. The data stack in AOC is exactly right for this, and it is easy to see why. The lifetimes of successive activations of any variable will be nested in time as the program runs. For example, we need the storage for X2 after we need it for X1, and we shall cease needing it before we cease needing it for X1. Clearly, a last-in-first-out stack for variables is appropriate.

Our first variable allocation scheme is based upon this observation. We shall arrange that when a block is entered, the previous values of all its local variables will be saved on the top of the current stack. When the procedure exits the old values will be "uncovered" and therefore restored, as the s-pointer is reset to its value before the call.

It is obviously expensive to make a complete copy of a procedure on each run-time call. We shall therefore write the return address on the stack, upon the call. Then we can return at the end to the location indicated by the stack value. If this were not saved on the stack, a recursive call would destroy the return address of the previous call.

We therefore need the following instructions for a procedure call and exit:

```

CP 1: Call procedure at location l
      s  $\leftarrow$  s + 1
      C(s)  $\leftarrow$  i {the present location}
      i  $\leftarrow$  l    {jump to the procedure}

RTN: Return
      i  $\leftarrow$  C(s) {the saved return address}
      s  $\leftarrow$  s - 1

```

The stacking and unstacking of local values can be done by an appropriate sequence of LV's at the block beginning and STD's at its end. We shall develop a more comprehensive plan later.

The above reversal program may then be translated as follows:

0	LC	0	{space for X}
1	BR	S	{branch around procedure code}
2	R LV	X	{save the old value}
3	READ		
4	STD	X	
5	LV	X	
6	LC	0	
7	EQ		
8	NOT		
9	JIF	L	
10	CP	R	
11	L LV	X	
12	PRINT		
13	STD	X	{restore the old value}
14	RTN		
15	S CP	R	
16	HALT		

Let the input tape contain {5, 6, 0}. Then the stack would appear as in figure 9.2 after the indicated instructions have been executed.

Each successive call of R reserves two locations for the return address (RA) and the previous value of X (shown as X0, X1, X2 in the figure). The return address word is called an *activation record* or *stack marker*. We shall later add more words to the stack marker, to support procedure calls in a more sophisticated manner.

A tentative block translation rule is therefore:

Translate each block independently, allocating space for its variables and its stack marker.

9.6.1. Procedures and the Free Variable Problem

There is an inconsistency between TRR and TPCR as they are presently formulated, and it is called the free variable problem. A *free variable* is a variable referenced in a procedure body but not local to the procedure. That is, it is declared outside the procedure block, but is in a block containing the procedure. For example, variable B in the following program is a free variable relative to procedure P:

Instructions executed	Stack														
0	s→ <table> <tr><td></td><td></td></tr> <tr><td>0</td><td>X</td></tr> </table>			0	X										
0	X														
1, 15, 2	s→ <table> <tr><td></td><td></td></tr> <tr><td>0</td><td>X0</td></tr> <tr><td>16</td><td>RA</td></tr> <tr><td>0</td><td></td></tr> </table>			0	X0	16	RA	0							
0	X0														
16	RA														
0															
3, 4, 5	s→ <table> <tr><td>3</td><td></td></tr> <tr><td>0</td><td>X0</td></tr> <tr><td>16</td><td>RA</td></tr> <tr><td>3</td><td>X</td></tr> </table>	3		0	X0	16	RA	3	X						
3															
0	X0														
16	RA														
3	X														
6, 7, 8, 9, 10	s→ <table> <tr><td></td><td></td></tr> <tr><td>11</td><td>RA</td></tr> <tr><td>0</td><td>X0</td></tr> <tr><td>16</td><td>RA</td></tr> <tr><td>3</td><td></td></tr> </table>			11	RA	0	X0	16	RA	3					
11	RA														
0	X0														
16	RA														
3															
2, 3	s→ <table> <tr><td></td><td></td></tr> <tr><td>6</td><td></td></tr> <tr><td>3</td><td>X1</td></tr> <tr><td>11</td><td>RA</td></tr> <tr><td>0</td><td>X0</td></tr> <tr><td>16</td><td>RA</td></tr> <tr><td>3</td><td></td></tr> </table>			6		3	X1	11	RA	0	X0	16	RA	3	
6															
3	X1														
11	RA														
0	X0														
16	RA														
3															
4, 5, 6, 7, 8, 9, 10	s→ <table> <tr><td></td><td></td></tr> <tr><td>11</td><td>RA</td></tr> <tr><td>3</td><td>X1</td></tr> <tr><td>11</td><td>RA</td></tr> <tr><td>0</td><td>X0</td></tr> <tr><td>16</td><td>RA</td></tr> <tr><td>6</td><td></td></tr> </table>			11	RA	3	X1	11	RA	0	X0	16	RA	6	
11	RA														
3	X1														
11	RA														
0	X0														
16	RA														
6															

Figure 9.2. Trace of AOC machines through execution of a program with a recursive procedure REV.

Instructions executed	Stack										
2, 3, 4, 5, 6, 7, 8	s→<table><tr><td></td></tr><tr><td>FALSE</td></tr><tr><td>6 X2</td></tr><tr><td>11 RA</td></tr><tr><td>3 X1</td></tr><tr><td>11 RA</td></tr><tr><td>0 X0</td></tr><tr><td>16 RA</td></tr><tr><td>0 X</td></tr><tr><td></td></tr></table>		FALSE	6 X2	11 RA	3 X1	11 RA	0 X0	16 RA	0 X	
FALSE											
6 X2											
11 RA											
3 X1											
11 RA											
0 X0											
16 RA											
0 X											
9, 11	s→<table><tr><td></td></tr><tr><td>0</td></tr><tr><td>6</td></tr><tr><td>11</td></tr><tr><td>3</td></tr><tr><td>11</td></tr><tr><td>0</td></tr><tr><td>16</td></tr><tr><td>0</td></tr></table>		0	6	11	3	11	0	16	0	
0											
6											
11											
3											
11											
0											
16											
0											
12, 13 (Print “0”)	s→<table><tr><td></td></tr><tr><td>11</td></tr><tr><td>3</td></tr><tr><td>11</td></tr><tr><td>0</td></tr><tr><td>16</td></tr><tr><td>6</td></tr></table>		11	3	11	0	16	6			
11											
3											
11											
0											
16											
6											
14, 11, 12, 13 (Print “0”)	s→<table><tr><td></td></tr><tr><td>11</td></tr><tr><td>0</td></tr><tr><td>16</td></tr><tr><td>3</td></tr></table>		11	0	16	3					
11											
0											
16											
3											

Figure 9.2. (cont'd.)

Instructions executed	Stack
14, 11, 12, 13 (Print "3")	s→ 16 0
14, 16 (Halt)	s→ 0

Figure 9.2. (cont'd.)

```

begin integer B;
  procedure P;
    begin B:=5
    end;
    B:=20;
    begin integer B;
      B := 7;
      P;
      print(B)
    end;
    print(B)
  end
end

```

Using TPCR we can arrive at the following program state:

```

begin integer B = 20;
  procedure P;
    begin B := 5
    end;
    B := 20;
    begin integer B=7;
      B := 7;
      → begin B:=5
        end;
        print(B)
      end
    end
  end
end

```

On the other hand, if we first apply TRR to the second use of B, and proceed as before we arrive at this state:

```

begin integer B = 20;
  procedure P;
    begin B := 5
    end;
    B := 20;
    begin integer B1 = 7;
      B1 := 7;
      →   begin B:=5
            end;
            print(B1)

```

The only difference is that the inner block declares B1, not B. It should be clear that the first simulation will print 5, 20, while the second will print 7, 5. TRR has been violated in the first simulation. Our implementation of this procedure as it stands will mimic the first, rather than the second simulation. Why this happens is not hard to see. When procedure P is called, its variable environment consists of the second B (B1), and this environment is not changed by the call mechanism. However, as the procedure appears in the source, its variable environment should consist of the first B, not the second, in order to preserve TRR. We see that a procedure call requires adjustments to the variable environment of the stack.

We clearly must also resolve the inconsistency between TPCR and TRR. We know that TRR is an important rule to have. It means that the writer of a procedure need only pay attention to the variables within his procedure and those in blocks that contain his procedure. He should not have to be concerned with the many and varied variable environments of the procedure calls. We therefore amend TPCR to require that it may not be applied until all independently declared variables have been made distinct through TRR.

Even this change is not good enough; it seems to break down for recursive procedures which seem to require application of TPCR first (to copy the procedure), then TRR. The situation gets quite complicated (see Kanner, in Rosen [1967], pages 228-252). We choose not to further develop the copying rules, but rather embark upon a revision of the procedure calls, blocks, and variable accessing instructions in order to preserve TRR.

9.6.2. Textual Addresses

We need a special naming system for variables in Algol programs. The *block level* (BL) of a block is the number of blocks outside it. The outer block therefore has BL = 0, the one immediately inside it has BL = 1, etc. The *textual address* (TA) of a variable is a pair of integers [B,j], where B is the BL of the block in which it was declared and j is an offset (measured in stack words) from a stack marker corresponding to the block of level B. For